1 Quantum Error Correction

The main challenge of quantum computation is the fragility of the physical systems used to implement qubits. In classical computation, information is encoded in macroscopic physical variables, and the logical states are separated by energy barriers that make them robust against environmental fluctuations. Compared to classical bits, qubits are heavily affected by environmental noise, which makes them degrade and lose their information due to decoherence. It is thus necessary to explore the most efficient ways to protect quantum information while maintaining the practical feasibility of operating with those protected qubits. [19]

1.1 Overview and Simple Codes

Fathoming the differences between classical and quantum error-corecting codes makes it necessary to pay attention to three difficulties that arise when we deal with quantum systems: The no-cloning theorem,, which prevents us from duplicating the state of a qubit; the continuous nature of errors; and the fact that observation in quantum mechanics collapses the wave function in one of the eigenstates of the measured observable (this is a non-unitary process, and thus recovery is impossible)

In order to overcome these difficulties, quantum error-correcting codes (QECC) utilize entanglement to encode a logical qubit across a multi-qubit system. This way, we can protect quantum information through redundancy, without violating the no-cloning theorem. QEC protocols measure multi-qubit parity operators (error syndromes), avoiding the collapse of the wave function. The continuity problem will be confronted by discretizing the errors into a finite set of Pauli operations.

We will build some basic intuition about quantum error correction models based on redundancy by looking at one of its earliest and simplest examples: the Shor Nine-Qubit Code.

Consider that we have physical qubits that are subject to random bit-flip errors, that is, with a certain probability p, the state $|\psi\rangle$ is transformed into $X|\psi\rangle$. As we cannot simply duplicate our original qubit, we can entangle it with two ancilla qubits initialized in the $|0\rangle$ state by applying two `CNOT` gates, where the original qubit is the control and the ancilla qubits are the targets. If our original state is $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, the resulting state will be $|\psi_L\rangle = \alpha|000\rangle + \beta|111\rangle$. The encoding establishes a logical basis with states:

$$|0_L\rangle = |000\rangle$$
$$|1_L\rangle = |111\rangle$$

If the bit-flip error occurs in any qubit, we can diagnose this error without destroying the superposition by measuring the parity observables Z_1Z_2 and Z_2Z_3. Those observables compare the parities of their corresponding qubits, yielding a ± 1 eigenvalue, positive if both compared qubits share the same state, negative if they are different. It is crucial to note that the measurement of the parity operators commutes with the logical operators, ensuring that the superposition remains intact. Once we have detected the error, we can

apply the appropiate recovery operation.

This same procedure can be adapted to protect against phase-flip errors, modeled by the Pauli Z operator. This is performed by applying a Hadamard transformation to each qubit after applying two sequential `CNOT` gates. It is an identical procedure, and it only differs in the basis we are working with.

If we concatenate the bit-flip and phase-flip codes, we can protect against an arbitrary error on a single qubit. This is therefore a nine-qubit code, with the base states defined as follows:

$$|0_L\rangle = \frac{1}{2\sqrt{2}}(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)$$
$$|1_L\rangle = \frac{1}{2\sqrt{2}}(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)$$

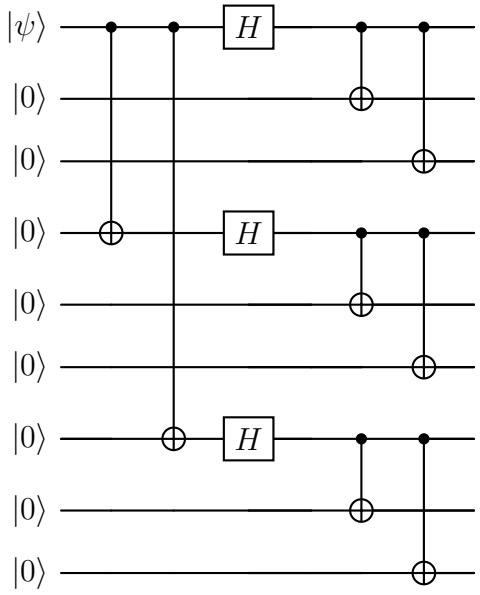

Figure 1: Shor Code circuit

The syndrome measurement is performed in an analogous way. We first compare the first two qubits by applying the operator $Z_1 Z_2$, then we continue comparing each pair of neighboring qubits by applying the corresponding operators. We can detect the bit-flip errors on any of the qubits. In order to detect phase-flip errors, we compare the signs of each of the three-qubit blocks.

The Shor code allows us to detect bit-flip and phase-flip errors. In order to illustrate that this condition is enough to detect an arbitrary error, we will describe the noise by a trace-preserving operation $\mathcal{E}$. Given ρ the density matrix of the system, the noise acting on the state can be expressed as:

$$\mathcal{E}(\rho) = \sum_k E_k \rho E_k^\dagger \tag{1}$$

This is called Kraus operator-sum representation, and will be discussed in more detail. The operators E_k can be expanded as a linear combination of the Pauli matrices $E_k = e_{i0}I + e_{i1}X_1 + e_{i2}Z_1 + e_{i3}X_1Z_1$. The resulting quantum state $E_k |\psi\rangle$ can be expressed as a superposition of $|\psi\rangle$, $X_1 |\psi\rangle$, $Z_1 |\psi\rangle$, and $X_1Z_1 |\psi\rangle$. When we measure the error syndrome, the system collapses to any of the four states, and after applying the necessary correction operation, we obtain the final state $|\Psi\rangle$.

1.2 General Theory of Quantum Error Correction

Once we have built some intuition about QEC, we can construct a more general theory.

Firstly, we need to give a mathematical description of our noise $\mathcal{E}$ [9]. Physically, noise represents an interaction between the system and the environment. Since we do not have access to the environment, the action of noise on the system appears to be a non-unitary operation, and we cannot describe our qubit as if it was in a pure state. This forces us to use the density matrix

$$\rho = \sum_i p_i |\psi_i\rangle \langle\psi_i| \tag{2}$$

that represents the mixture of different pure states $|\psi_i\rangle$ with probabilities p_i. This must be a positive semi-definite Hermitian matrix, with unit trace and non-negative eigenvalues. Non-unitary noise operations must preserve these properties. Hence, we represent noise acting on a system as a transformation of the state described by a density matrix that fulfills this condition: a Completely Positive Trace Preserving (CPTP) map. In the Kraus representation, the action of a CPTP map $\mathcal{E}$ on a density matrix ρ is given by the formula 1, where the Kraus operators E_k satisfy the condition $\sum_k E_k^\dagger E_k = I$ (this condition derives necessarily from the requirement of trace preservation). If we consider any state $|\psi\rangle$ in the Hilbert space and define the states $|\psi_k\rangle = E_k^\dagger |\psi\rangle$, we can easily check that:

$$\sum_k \langle\psi| E_k \rho E_k^\dagger |\psi\rangle = \sum_k \langle\psi_k| \rho |\psi_k\rangle \geq 0$$

which verifies that the map is positive. In fact, Choi's theorem on completely positive maps guarantees that any quantum operation admitting such a Kraus representation is inherently *completely positive*, meaning it yields valid physical states even when applied to a subsystem of a larger entangled system. We will call the operation elements $\{E_i\}$ errors, and they represent the transformation of the quantum state under Hamiltonian parameter deviations from their ideal values.

Now we want to generalize the basic ideas present in the Shor code to give a simple picture of how we can detect and correct errors in a quantum system. We firstly define a quantum error-correcting code as a subspace C of the total Hilbert space, and the elements of C will be called codewords. We encode our quantum states by projecting them into C by a unitary operation P that we will call the projector. The encoded system is then vulnerable to environmental noise, which we model as a CPTP map $\mathcal{E}$. We perform

a syndrome measurement to determine the error that has affected the system, and a recovery operation $\mathcal{R}$ returns the system to its original encoded state.

When we project our state into the code subspace, we are essentially restricting the degrees of freedom of the system. This means that if our original Hilbert space for n physical qubits is 2^n-dimensional, the code subspace will be 2^k-dimensional, where $k < n$. We will call C a $[n, k]$ code, as we are encoding k logical qubits into n physical qubits. Once the noise acts on the encoded state $|\psi_L\rangle$, the system will be pushed into a different subspace. Specifically, each error E_α projects the state into an error subspace $C_\alpha = E_\alpha C$. Each error subspace has its corresponding projector $P_\alpha = E_\alpha P E_\alpha^\dagger$.

If, given a code, each error corresponds to different undeformed and orthogonal subspaces, then we can reliably distinguish which error has occurred. This will be achieved by performing the set of projective measurements associated with each error subspace. Given our noisy state $E_\alpha |\psi_L\rangle$, the execution of a projector P_β will yield us a syndrome lecture β with probability $p_\beta = \langle\psi_L| E_\alpha^\dagger P_\beta E_\alpha |\psi_L\rangle$. The orthogonality of error subspaces ensures that the measurement associated to an error subspace C_β will only yield a non-zero probability when the corresponding error has occurred.

The undeformed condition means that, if you have two perfectly distinguishable (orthogonal) states $|\psi_L\rangle$ and $|\phi_L\rangle$ in the code subspace, then the error E_α must not make them non orthogonal. This is equivalent to saying that the inner product of the two states must be preserved under the action of the error, that is, $\langle\psi_L| E_\alpha^\dagger E_\alpha |\phi_L\rangle = \langle\psi_L|\,|\phi_L\rangle = 0$. This ensures that the recovery operation can correctly restore the original encoded state.

We can define the condition for an error correction code to be able to correct the effect of noise as follows: Given a code C, a noisy quantum channel characterized by $\mathcal{E}$, and any state $|\psi_L\rangle$, with density matrix ρ_L, the error correction is successful if

$$(\mathcal{R} \circ \mathcal{E})(\rho) \propto \rho \tag{3}$$

In this condition, and in the following equation, we have not imposed the trace-preserving condition for $\mathcal{E}$, with the idea of allowing for more general noise models. That is the reason we use the proportionality symbol $\propto$. The prior discussion can be summarized in a very compact algebraic expression, the Knill-Laflamme condition for quantum error correction [16]:

Theorem 1 (Knill-Laflamme)**.** *Let C be a quantum error-correcting code, P the projector onto C, and $\mathcal{E}$ a CPTP map with operation elements $\{E_i\}$. Then, there exists a recovery operation $\mathcal{R}$ such that $(\mathcal{R} \circ \mathcal{E})(\rho) = \rho$ for all ρ supported on C if and only if the following condition holds:*

$$P E_i^\dagger E_j P = \alpha_{ij} P \tag{4}$$

for some Hermitian matrix α_{ij}.

Proof. ($\Leftarrow$) We notice that the Hermitian matrix α_{ij} can be diagonalized by a unitary transformation u, such that $u\alpha u^\dagger = d$, where d is a diagonal matrix. We can define new operation elements $F_i = \sum_j u_{ij} E_j$. Those can be proven to be operation elements of $\mathcal{E}$.

The new operation elements also satisfy the Knill-Laflamme condition, since:

$$\begin{aligned} PF_i^\dagger F_j P &= \sum_{k,l} u_{ki}^\dagger u_{jl} P E_k^\dagger E_l P \\ &= \sum_{k,l} u_{ki}^\dagger u_{jl} \alpha_{kl} P \\ &= d_{kl} P \end{aligned}$$

which simplifies the Knill-Laflamme condition, since d_{kl} is a diagonal matrix. This diagonalization makes it explicit that the error subspaces are orthogonal. We decompose the operation elements as $F_i P = \sqrt{d_{ii}} U_i P$, where U_i is a unitary operator and $\sqrt{d_{ii}}$ is a positive real number.

F_i maps the code subspace C to the error subspace defined by the projector $P_i = U_i P U_i^\dagger = F_i P U_i^\dagger / \sqrt{d_{ii}}$. As we have already discussed, the syndrome measurement is the projective measurement defined by the projectors P_i. The recovery operation $\mathcal{R}$ is defined as $\mathcal{R}(\rho) = \sum_k U_i^\dagger P_i \rho P_i U_i$. For any state ρ supported on C, we can show that:

$$\begin{aligned} U_i^\dagger P_i F_k \sqrt{\rho} &= U_i^\dagger P_i^\dagger F_j P \sqrt{\rho} \\ &= \frac{U_i^\dagger U_i P F_i^\dagger F_j P \sqrt{\rho}}{\sqrt{d_{ii}}} \\ &= \delta_{ij} \sqrt{d_{ij}} \sqrt{\rho} \end{aligned}$$

Thus, one can easily check that this recovery operation satisfies the error correction condition:

$$\begin{aligned} (\mathcal{R} \circ \mathcal{E})(\rho) &= \sum_{i,j} U_i^\dagger P_i F_j \rho F_j^\dagger P_i U_i \\ &= \sum_{i,j} \delta_{ij} d_{ii} \rho \propto \rho \end{aligned}$$

($\Rightarrow$) Assume $\{E_i\}$ are errors correctable by $\mathcal{R}$, with operation elements $\{R_j\}$. We define a quantum operation $\mathcal{E}_C$ by $\mathcal{E}_C(\rho) = \mathcal{E}(P\rho P)$. $P\rho P \in C$ for any ρ, so we can apply the error correction condition to obtain:

$$\mathcal{R}(\mathcal{E}_C(\rho)) \propto P\rho P$$

We can rewrite the left-hand side of this equation as:

$$\mathcal{R}(\mathcal{E}_C(\rho)) = \sum_{i,j} R_j E_i P \rho P E_i^\dagger R_j^\dagger$$

Thus

$$\sum_{i,j} R_j E_i P \rho P E_i^\dagger R_j^\dagger = c P \rho P$$

for some constant c. The quantum operation with operation elements $\{R_j E_i\}$ is identical to the quantum operation with only one operation element $\sqrt{c}P$. This implies that there exist a complex matrix with entries c_{ki} such that $R_k E_i P = c_{ki} P$. We take the adjoint and obtain $P E_i^\dagger R_k^\dagger R_k E_j P = c_{ki}^* c_{kj} P$. Since $\mathcal{R}$ is a CPTP map, we have $\sum_k R_k^\dagger R_k = I$. Thus, we can sum over k to obtain:

$$P E_i^\dagger E_j P = \alpha_{ij} P$$

Which is the Knill-Laflamme condition, with $\alpha_{ij} = \sum_k c_{ki}^* c_{kj}$.

□

The proof of the Knill-Laflamme condition explicitly provides a method to construct the recovery operation $\mathcal{R}$, given the code subspace and the noise model. However, a quantum system could be affected by a noise process that is not known. It would be desirable to have a code C and an error-correction operation $\mathcal{R}$ that can correct the effect of an entire class of noise processes. That is indeed possible, and it is a consequence of the linearity of quantum mechanics. If we have a code that can correct the effect of a set of errors $\{E_i\}$, then it can also correct the effect of any error that can be expressed as a linear combination of those errors. This is a direct consequence of theorem 1, since if C satisfies the Knill-Laflamme condition for $\{E_i\}$, then it also satisfies it for any linear combination of those errors.

1.3 Stabilizer Codes

The previous discussion has proven the theoretical possibility of quantum error correction. However, the state-vector representation of quantum states carries a practical impossibility. When we have a a large number n of physical qubits, the manipulation of the 2^n-dimensional state vector becomes unfeasible. We need a more efficient way to represent quantum states, and the stabilizer formalism provides us with such a tool, as well as a method to construct QECC.

1.3.1 Stabilizer Formalism

If we take a closer look at the Shor code, and specifically at the syndrome measurement, we can see that in order to detect a bit flip on one of the first three qubits, we measured the eigenvalues of the operators $Z_1 Z_2$ and $Z_2 Z_3$. The same procedure should be performed for the other two blocks of three qubits. In order to detect phase-flip errors, we need to measure the eigenvalues of the operators $X_1 X_2 X_3 X_4 X_5 X_6$ and $X_4 X_5 X_6 X_7 X_8 X_9$. In total, we need to measure the eigenvalues of eight operators.The two valid codewords, $|0_L\rangle$ and $|1_L\rangle$, are the eigenstates of all these operators with eigenvalue $+1$, and hence are simulatenously fixed by any of them. Those are called stabilizer operators, and the basic idea of the formalism is to describe the code space as the subspace stabilized by these operators.

In order to give a more precise description of the stabilizer formalism, let's introduce the n-dimensional Pauli group $\mathcal{P}_n$, which is the group generated by the n-fold tensor products of the Pauli matrices and the identity, together with the multiplicative factors ± 1 and $\pm i$. For instance, the 1-dimensional Pauli group is

$$\mathcal{P}_1 = \{\pm I, \pm X, \pm Y, \pm Z, \pm iI, \pm iX, \pm iY, \pm iZ\}$$

It is easily verifiable that $\mathcal{P}_n$ is a group under matrix multiplication. The stabilizer is some Abelian subgroup $\mathcal{S} \subseteq \mathcal{P}_n$ that does not contain $-I$, and the vectors fixed by all elements of $\mathcal{S}$ form the code space $C(\mathcal{S})$, i.e.:

$$C(\mathcal{S}) \equiv \{|\psi\rangle \in \mathcal{H} : g\,|\psi\rangle = |\psi\rangle\,;\ \forall g \in \mathcal{S}\}$$

It is also obvious that the code space $C(\mathcal{S})$ is indeed a vector space. $\mathcal{S}$ is necessarily an Abelian subgroup, since two non-conmuting operators cannot have common eigenstates.

Let's recall that our aim is to define a subspace C of the total Hilbert space, such that it encodes k logical qubits into n physical qubits. Therefore, the dimension of C must be 2^k. We want to see how this condition translates on the definition of the stabilizer $\mathcal{S}$:

Proposition 1. *If $\mathcal{S}$ is an Abelian subgroup of $\mathcal{P}_n$ that does not contain $-I$, then the code space $C(\mathcal{S})$ stabilized by $\mathcal{S}$ has dimension 2^{n-k}, where k is the number of independent generators of $\mathcal{S}$.*

Proof. Let $\mathcal{S}$ be generated by $\langle g_1, g_2, \ldots, g_k\rangle$. Since all the generators are elements of $\mathcal{P}_n$, they are unitary and Hermitian, and thus have eigenvalues ± 1. We can project the total Hilbert space into the $+1$ eigenspace of each generator g_i by applying the projector

$$P_i = \frac{I + g_i}{2}$$

The code space $C(\mathcal{S})$ is the subspace stabilized by all the generators, and thus can be obtained by applying the projectors associated with each generator to the total Hilbert space:

$$P_C = \prod_{i=1}^{k} P_i = \prod_{i=1}^{k} \frac{I + g_i}{2}$$

If we expand the product, we sum over all possible products of the generators g_i, which yields the 2^k elements of $\mathcal{S}$.

$$P_C = \frac{1}{2^k} \sum_{g \in \mathcal{S}} g$$

The dimension of the code space $C(\mathcal{S})$ is given by the trace of the projector P_C:

$$\dim(C(\mathcal{S})) = \mathrm{Tr}(P_C) = \frac{1}{2^k} \sum_{g \in \mathcal{S}} \mathrm{Tr}(g)$$

We have the following cases for the trace of an element $g \in \mathcal{S}$:

- If $g = I^{\otimes n}$, then $\mathrm{Tr}(g) = 2^n$.

- If $g \neq I^{\otimes n}$, then g is a non-trivial element of $\mathcal{P}_n$, and thus has trace zero, since it contains at least one Pauli matrix that has zero trace.

$-I \notin \mathcal{S}$ ensures that there are no elements of $\mathcal{S}$ with trace -2^n. Thus, the only element of $\mathcal{S}$ with non-zero trace is the identity, and we have:

$$\dim(C(\mathcal{S})) = \mathrm{Tr}(P_C) = \frac{1}{2^k} \cdot 2^n = 2^{n-k}$$

□

Therefore, a code that encodes k logical qubits into n physical qubits can be defined by a stabilizer $\mathcal{S}$ with $n-k$ independent generators. This proposition also provides a reason for the non-inclusion of $-I$ in the stabilizer. This is nonetheless obvious from the fact that the only solution to $-I|\psi\rangle = |\psi\rangle$ is the trivial solution $|\psi\rangle = 0$.

1.3.2 Unitary Dynamics of Stabilizer Codes and the Gottessman-Knill Theorem

We have discussed the description of a subspace using stabilizer formalism. Now let's see how we can describe the dynamics of those subspaces under the action of noise and other quantum processes. Let $|\psi\rangle$ be an element of a code space $C(\mathcal{S})$ stabilized by $\mathcal{S}$. If we apply a unitary operation U,

$$U|\psi\rangle = Ug|\psi\rangle = (UgU^\dagger)U|\psi\rangle \tag{5}$$

and we see that $U|\psi\rangle$ is stabilized by the conjugated operator $UgU^\dagger$. Since this is common to any element of $C(\mathcal{S})$, we can easily see that $UC(\mathcal{S})$ is stabilized by $USU^\dagger \equiv \{UgU^\dagger : g \in \mathcal{S}\}$. This property does not ensure that any stabilizer of a subspace $UC(\mathcal{S})$ is a valid stabilizer, since the conjugation by U might introduce elements that are not in $\mathcal{P}_n$. We hence introduce the Clifford group $\mathcal{C}_n$ as the normalizer of the Pauli group:

$$\mathcal{C}_n \equiv \{U : U\mathcal{P}_n U^\dagger = \mathcal{P}_n\}$$

The Clifford group is the largest group of unitary operations that preserve the Pauli group under conjugation. If we apply an operation $U \in \mathcal{C}_n$ to a code space C stabilized by $\mathcal{S}$, the resulting subspace UC is stabilized by $USU^\dagger \in \mathcal{P}_n$. Conjugation by a Clifford operation also preserves the commutation relations

$$Ug_1U^\dagger Ug_2U^\dagger = Ug_1g_2U^\dagger = Ug_2g_1U^\dagger = Ug_2U^\dagger Ug_1U^\dagger$$

which ensures that the resulting stabilizer $USU^\dagger$ is also an Abelian subgroup of $\mathcal{P}_n$. If $-I$ was in $U\mathcal{S}U^\dagger$, then there would be some $g \in \mathcal{S}$ such that $UgU^\dagger = -I$, and therefore:

$$U^\dagger(UgU^\dagger)U = U^\dagger(-I)U$$
$$g = -I$$

which contradicts the definition of stabilizer. Thus, $-I$ is not in $U\mathcal{S}U^\dagger$, and the resulting subspace UC is a valid code space stabilized by $USU^\dagger$.

The Clifford group contains the Pauli group, but is larger. $\mathcal{C}_n$ is generated by the Hadamard gate, the phase gate, and the `CNOT` gate. It is important to note that this small set is enough to create highly entangled states.

This formalism provides us with a method to efficiently represent the dynamics of quantum circuits under the action of Clifford operations. The idea is that, instead of representing the state of the system as a vector in a 2^n-dimensional Hilbert space, we can represent it by its stabilizer, which is generated by n independent generators. Each generator is an element of $\mathcal{P}_n$, and thus can be represented by a string of n characters, each of which can be I, X, Y, or Z. Therefore, we can represent the state of the system by a list of n strings of length n. This discussion culminates in the following theorem:

Theorem 2 (Gottesman-Knill)**.** *Any quantum circuit that consists solely of Clifford gates, preparation of qubits in the computational basis, and measurements of Pauli operators can be efficiently simulated on a classical computer.*

The Clifford group is not universal for quantum computation, and thus the Gottesman-Knill theorem does not contradict the fact that quantum computers can outperform classical computers. However, it is important to note that the Clifford group is a very large group of operations, and it is enough to create highly entangled states, and many quantum error correction circuits, as well as quantum protocols such as teleportation or superdense coding can be implemented using only Clifford gates. In order to achieve universality, we need to add at least one non-Clifford gate, such as the T gate.

1.3.3 Error Detection and Correction in Stabilizer Codes

Now that we have established the stabilizer formalism, we can explicitly describe how a stabilizer code detects and corrects errors in a remarkably elegant way. [10]

Let $C(\mathcal{S})$ be an $[n, k]$ stabilizer code defined by the stabilizer group $\mathcal{S} = \langle g_1, g_2, \ldots, g_{n-k} \rangle$. A valid encoded state $|\psi_L\rangle \in C(\mathcal{S})$ satisfies $g_i |\psi_L\rangle = |\psi_L\rangle$ for all $i \in \{1, \ldots, n-k\}$.

Suppose an arbitrary error occurs. As discussed in Section 1.1, we can discretize errors by expanding them in the basis of the Pauli group $\mathcal{P}_n$. Thus, we only need to consider what happens when a Pauli error $E \in \mathcal{P}_n$ corrupts our state, transforming it into $E |\psi_L\rangle$. To detect the error without measuring the logical state (which would destroy the quantum information), we measure the $n-k$ stabilizer generators g_i.

Since both E and g_i belong to the Pauli group $\mathcal{P}_n$, they must either commute ($Eg_i = g_iE$) or anti-commute ($Eg_i = -g_iE$). This fundamental property of the Pauli group dictates the outcome of our syndrome measurements:

- If E commutes with g_i: $g_i(E |\psi_L\rangle) = Eg_i |\psi_L\rangle = E |\psi_L\rangle$. The corrupted state is an eigenstate of g_i with eigenvalue $+1$.
- If E anti-commutes with g_i: $g_i(E |\psi_L\rangle) = -Eg_i |\psi_L\rangle = -E |\psi_L\rangle$. The corrupted state is an eigenstate of g_i with eigenvalue -1.

The outcomes of these $n-k$ measurements form the *error syndrome*. Typically, we map the eigenvalues ± 1 to binary values $s_i \in \{0, 1\}$, where $s_i = 0$ corresponds to

$+1$ (commutes) and $s_i = 1$ corresponds to -1 (anti-commutes). The syndrome vector $\vec{s} = (s_1, s_2, \ldots, s_{n-k})$ provides a unique fingerprint for the error E, provided that the code is capable of correcting it.

Once the syndrome $\vec{s}$ is extracted, classical processing (decoding) is used to determine the most likely error E that occurred. The correction step simply consists of applying a recovery operator $R \in \mathcal{P}_n$ that shares the same syndrome as E. If the decoding is successful, RE will commute with all stabilizers, and ideally $RE \in \mathcal{S}$. Since elements of $\mathcal{S}$ act as the identity on the code space, the combined effect of the error and the recovery operation is trivial, successfully restoring the original state: $RE\,|\psi_L\rangle = |\psi_L\rangle$.

However, not all errors can be successfully corrected. If an error E commutes with all stabilizer generators but is not an element of $\mathcal{S}$, it will yield a trivial syndrome (all $+1$) while transforming the state into a different, orthogonal codeword. The set of all operators in $\mathcal{P}_n$ that commute with all elements of $\mathcal{S}$ is the centralizer of $\mathcal{S}$, which for the Pauli group is equivalent to the normalizer $N(\mathcal{S})$.

Therefore, undetectable and uncorrectable logical errors are exactly those in $N(\mathcal{S}) \setminus \mathcal{S}$. The minimum weight (defined as the number of non-identity tensor factors in the Pauli string) of an operator in $N(\mathcal{S}) \setminus \mathcal{S}$ defines the distance d of the code. Such a code is usually denoted by the parameters $[n, k, d]$, meaning it encodes k logical qubits into n physical qubits and can correct any arbitrary error affecting up to $\lfloor (d-1)/2 \rfloor$ physical qubits.

We can build a deeper structural understanding of the code by considering the logical characterisation of errors instead of just tracking them as a raw collection of physical errors. This means that we can find different elements of $N(\mathcal{S}) \setminus \mathcal{S}$ that act on the encoded states in the same way. Specifically, for any given error $E \in N(\mathcal{S}) \setminus \mathcal{S}$, we can find an element $g \in \mathcal{S}$ such that $Eg \in N(\mathcal{S}) \setminus \mathcal{S}$, and $Eg\,|\psi_L\rangle = E\,|\psi_L\rangle$ for any $|\psi_L\rangle \in C(\mathcal{S})$. This means that we could build a group of logical operators where different physical errors differing only by elements of $\mathcal{S}$ are identified in the same class of equivalence. This is obviously the definition of the quotient group $N(\mathcal{S})/\mathcal{S}$. The elements of this group are also the valid logical operators of the code, as they are the only operators that act non-trivially on the encoded states without pushing them out of the code space.

1.4 Fault-Tolerant Quantum Computation

We have discussed how encoding quantum information into a subspace can protect the data from environmental noise. However, the very process of encoding, syndrome measurement, as well as the application of logical quantum gates can also be affected by noise. The fundamental idea of fault-tolerant quantum computation is to design these processes in such a way that they do not propagate errors uncontrollably, and that the error correction procedure can still successfully correct the errors that occur during these operations.

Multi-qubit gates carry the risk of propagating errors from one physical qubit to another. The reiterative application of such operations can produce a single physical hardware failure to cascade into a large number of errors on the physical qubits encoding

the logical state. If the weight of the resulting error exceeds the distance d of the code, the error correction procedure will fail. Therefore, the design of fault-tolerant quantum circuits must ensure that errors do not propagate in a way that exceeds the error-correcting capabilities of the code.

We can prevent catastrophical error propagation by imposing the condition of transversality on the implementation of logical gates. A transversal gate is one that can be implemented by applying independent physical operations exclusively between corresponding physical qubits in different code blocks. This architectural method ensures that a single physical error propagates to at most one physical qubit in each code block, thus maintaining the error weight within the correctable treshold of the code. This may seem like a very simple condition, but it has a profound limitation [7]:

Theorem 3 (Eastin-Knill)**.** *No local quantum error-correcting code can implement a universal set of logical gates transversally.*

This theorem implies that the implementation of transversal gates alone does not suffice to achieve universal fault-tolerant quantum computation.

2 Topological Quantum Error Correction

We have introduced the algebraic framework for diagonsing and correcting quantum errors. However, the practical implementation of QECC must face another critical challenge: the strict near-neighbor connectivity of physical qubits in real scalable hardware platforms, such as superconducting circuits [1] [11]. This constraint necessitates the development of quantum error-correcting codes that can be implemented with local interactions, i.e., families of stabilizer codes where the number of qubits in the support of any of the stabilizer generators must be bounded. This will be the depart point of topological quantum codes, a concept initiated by Kitaev's toric code [15].

2.1 Homology

We are interested in a specific family of stabilizer codes, namely surface codes. Those can be regarded as encoding quantum information in the homological degrees of freedom of a two-dimensional lattice. In order to understand the construction of surface codes, we need to introduce some concepts from algebraic topology [3].

Let's consider a connected, closed, orientable surface of genus g, and define a lattice on it, with V vertices. E edges and F faces. We will call 0-cells the vertices of the lattice, 1-cells the edges, and 2-cells the faces.

We want to obtain the homology groups of this surface over $\mathbb{Z}_2$. For this purpose, we construct an abelian group for each dimension $i \in \{0, 1, 2\}$, generated by the sets of i-cells, where the field of coefficients is $\mathbb{Z}_2$. In this sense, if we consider a set of 1-cells $E = \{e_1, e_2, \ldots, e_m\}$, we can represent it as a sum

$$c = \sum_i c_i e_i \qquad c_i = \begin{cases} 0 & e_i \notin E \\ 1 & e_i \in E \end{cases}. \tag{6}$$

Such sums are called 1-chain. We can consider addition of 1-chains (taking into account that the coefficients are in $\mathbb{Z}_2$ and hence $e_i + e_i = 0$), obtaining an abelian group structure on the set of 1-chains C_1. The elements of C_1 can be considered as binary vectors of length E, and topologically $C_1 \simeq \mathbb{Z}_2^E$. Equivalently, we define the group of 0-chains $C_0 \simeq \mathbb{Z}_2^V$ as the abelian group generated by the sets of 0-cells, and the group of 2-chains $C_2 \simeq \mathbb{Z}_2^F$ using the exact same procedure for 2-cells.

The entire homology theory rests on the definition of the boundary operators $\partial_i : C_i \to C_{i-1}$, which are a family of group homomorphisms that take an i-chain and return its geometric boundary as an $(i-1)$-chain. Given our construction, we are interested in two boundary operators:

- $\partial_1 : C_1 \to C_0$ takes a 1-chain (a set of edges) and returns the set of vertices that are the endpoints of an odd number of edges in the 1-chain. For instance, if we have a 1-chain $c = e_1 + e_2 + e_3$, where e_1 connects vertices v_1 and v_2, e_2 connects vertices v_2 and v_3, and e_3 connects vertices v_3 and v_4, then $\partial_1(c) = v_1 + v_4$, since v_1 and v_4 are the endpoints of an odd number of edges in c, while v_2 and v_3 are the endpoints of an even number of edges in c.
- $\partial_2 : C_2 \to C_1$ takes a 2-chain (a set of faces) and returns the set of edges that are the endpoints of an odd number of faces in the 2-chain.

The boundary operators satisfy the property that they are nilpotent of degree 2, meaning that $im(\partial_{i+1}) \subseteq ker(\partial_i)$, or equivalently $\partial_i \circ \partial_{i+1} = 0$. This property is a direct consequence of the geometric interpretation of the boundary operators, since the boundary of a boundary is always empty.

From these boundary operators, we construct two subgroups of C_i. The group of i-cycles Z_i is defined as the kernel of ∂_i, that is, the set of i-chains that have an empty boundary. The group of i-boundaries B_i is defined as the image of ∂_{i+1}, that is, the set of i-chains that are the boundary of some $(i+1)$-chain. Since $\partial_i \circ \partial_{i+1} = 0$, we have that $B_i \subseteq Z_i$.

We can define the i-th homology group as the algebraic quotient group of i-cycles by i-boundaries:

$$H_i = \frac{Z_i}{B_i} \tag{7}$$

where the quotient group is defined as the set of equivalence classes of i-cycles, where two i-cycles are equivalent if they differ by a i-boundary, that is, the elements of H_i are of the form $[z] = \{z + b : b \in B_i\}$, where z is any i-cycle. The homology groups are topological invariants, meaning that they do not depend on the specific lattice we have defined on the surface, but only on the topology of the surface itself. For a connected, closed, orientable surface of genus g, we have that $H_0 \simeq \mathbb{Z}_2$, $H_1 \simeq \mathbb{Z}_2^{2g}$ and $H_2 \simeq \mathbb{Z}_2$.

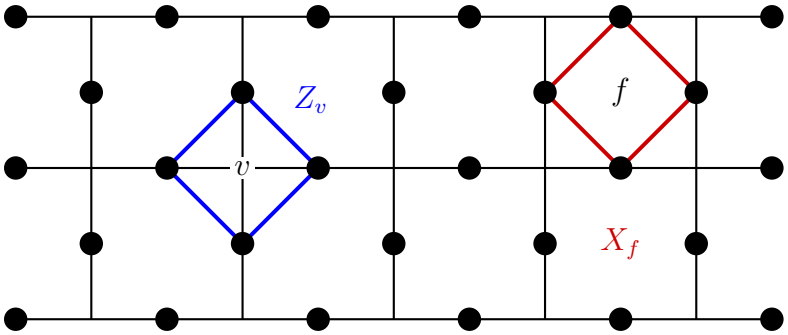

Figure 2: Lattice showing the arrangement of physical qubits and the support of the stabilizer generators.

2.2 Surface Codes

Consider a lattice embedded in a closed surface, and attach a qubit to each edge. It is natural to see elements of the computational basis as 1-chains $c \in C_1$

$$|c\rangle = \bigotimes_i |c_i\rangle \tag{8}$$

It is useful to define groups homeomorphism from C_1 to the group of Pauli operators $\mathcal{P}_n$, where n is the number of edges in the lattice and hence the number of physical qubits. We can define two homomorphisms, $X : C_1 \to \mathcal{P}_n$ and $Z : C_1 \to \mathcal{P}_n$, such that

$$X_c = \bigotimes_i X_i^{c_i} \quad \text{and} \quad Z_c = \bigotimes_i Z_i^{c_i}. \tag{9}$$

Given these homomorphisms, we can define the face (or plaquette) and vertex (or star) operators as follows:

$$X_f := \prod_{e \in \partial_2 f} X_e \quad \text{and} \quad Z_v := \prod_{e | v \in \partial_1 e} Z_e \tag{10}$$

Those operators are ilustrated in figure 2, giving a clear intuition of their geometric meaning. The face and vertex operators commute with each other, and hence we can consider a stabilizer group $\mathcal{S}$ generated by them. In order to see the code subspace stabilized by $\mathcal{S}$, we can apply the projectors associated with each generator to any state of the total Hilbert space $|c\rangle$, $c \in C_1$:

$$\prod_f \frac{I + X_f}{2} \prod_v \frac{I + Z_v}{2} |c\rangle$$

Let's look at how each of the projectors acts on our arbitrary 1-chain state $|c\rangle$. The vertex operator Z_v effectively checks the parity of the edges of the chain c connected to the vertex v:

$$Z_v |c\rangle = (-1)^k |c\rangle \quad \text{where } k = |\{e \mid v \in \partial_1 e,\ e \in c\}|$$

From the definition of the boundary operator ∂_1, we can see that $Z_v |c\rangle = |c\rangle$ if and only if v is the endpoint of an even number of edges in c and hence $v \notin \partial_1 c$. If $v \in \partial_1 c$,

then $Z_v |c\rangle = -|c\rangle$ and the projector $(I + Z_v)|c\rangle = 0$. This means that the only states that survive the application of all vertex projectors are those associated with 1-cycles, $|z\rangle$, with $z \in Z_1$.

Now we apply the face projectors to our surviving states $|z\rangle$:

$$\prod_f \frac{1 + X_f}{2} |z\rangle$$

We can expand the product of all face terms and obtain a sum over all posible subsets of faces $\{f_i\}$. We also use that applying face operators on adjacent faces cancels out the operators on their shared edges:

$$\prod_f (1 + X_f) = \sum_{\{f_i\}} \prod_i X_{\partial_2 f_i} = \sum_{\{f_i\}} X_{\partial_2 (\sum_i f_i)}$$

We notice that $\sum_i f_i$ is a 2-chain, and thus $\partial_2(\sum_i f_i)$ is a 1-boundary. Therefore this is proportional to the sum over all possible 1-boundaries:

$$\prod_f (1 + X_f) \propto \sum_{b \in B_1} X_b$$

Applying this to our cycle state $|z\rangle$ yields:

$$\prod_f \frac{1 + X_f}{2} |z\rangle \propto \sum_{b \in B_1} X_b |z\rangle = \sum_{b \in B_1} |z + b\rangle \tag{11}$$

This means that the states associated with two different cycles differing by a boundary are projected to the same state, and hence the code subspace stabilized by $\mathcal{S}$ is the vector space generated by the states $|[z]\rangle = \sum_{b \in B_1} |z + b\rangle$, where $[z]$ is the equivalence class of z in H_1. Since $H_1 \simeq \mathbb{Z}_2^{2g}$, we have that the code subspace has dimension 2^{2g}, and hence encodes $k = 2g$ logical qubits into n physical qubits.

Now, let's consider the effects of bit-flip and phase-flip errors on the code subspace. We can model bit-flip errors as the application of X operators on the elements of a certain 1-chain c, so that its action on the physical state is given by $X_c |b\rangle = |b + c\rangle$, and on the encoded state $X_c |[z]\rangle = |[z + c]\rangle$. If the error chain is an open string ($\partial_1 c \neq 0$), then the error kicks the encoded state out of the protected subspace, so it can be easily detected. This means that X_z can map the code back to itself only if $z \in Z_1$. On the other hand, if the error acts on a 1-boundary $b \in B_1$, then the encoded state remains the same as the homology class does not change $X_b |[z]\rangle = |[z]\rangle$. We conclude that the distance of a surface code for bit-flip errors is the length of the shortest non-bounding cycle $z \in Z_1 \setminus B_1$.

In order to proceed with the analysis of phase-flip errors, we need to construct the dual lattice. We will map faces to dual vertices f^*, edges to dual edges e^* (they represent the same physical qubits) and vertices to dual faces v^*. We have the corresponding dual

operators $\partial_1^* : C_0^* \to C_1^*$ and $\partial_2^* : C_1^* \to C_2^*$, and they produce the groups of dual cycles Z_1^*, dual boundaries B_1^* and the homology group H_1^*. We can map the original code subspace to the one defined by the surface code on the dual lattice by applying a Hadamard gate $H^{\otimes E}$ to all physical qubits, giving us the following stabilizers:

$$H^{\otimes E} X_f H^{\otimes E} = \prod_{e \in \partial_2 f} Z_e = \prod_{e^* | f^* \in \partial_2^* e^*} Z_e =: Z_{f^*} \tag{12}$$

$$H^{\otimes E} Z_v H^{\otimes E} = \prod_{e | v \in \partial_1 e} X_e = \prod_{e^* \in \partial_1^* v^*} X_e =: X_{v^*}. \tag{13}$$

We can then treat phase-flip errors as bit-flip errors on the dual lattice, and applying the same reasoning as before, we conclude that the distance of a surface code for phase-flip errors is the length of the shortest non-bounding cycle $z^* \in Z_1^* \setminus B_1^*$ on the dual lattice.

If we recall the discussion on error detection in stabilizer codes, we can establish a clear parallelism between the normalizer and stabilizer groups, and the different 1-chain groups defined on the lattice. Firstly, we notice that we can write any element of the Pauli group $\mathcal{P}_E$ in terms of a pair of chains c, c^*, and a phase, as follows:

$$A = i^k X_c Z_{c^*} \tag{14}$$

where $k \in \{0, 1, 2, 3\}$ is an integer that depends on the specific element of $\mathcal{P}_E$. We want to find the elements of the normalizer, that is, operators A that commute with all the vertex and face operators. As we have already discussed, the commutation relations are:

$$[X_c, S_v] = 0 \iff v \notin \partial_1 c \qquad [Z_{c^*}, S_f] = 0 \iff f \notin \partial_2^* c^* \tag{15}$$

This means that the normalizer is given by the set of operators $A = i^k X_c Z_{c^*}$ such that $\partial_1 c = 0$ and $\partial_2^* c^* = 0$, which is equivalent to saying that $c \in Z_1$ and $c^* \in Z_1^*$. The stabilizer is given by the subset of the normalizer where $c \in B_1$ and $c^* \in B_1^*$. Therefore, the logical operators are given by the elements of the normalizer that are not in the stabilizer, which correspond to pairs of cycles that are not boundaries. This is exactly the definition of the homology groups H_1 and H_1^*, and hence we have that the logical operators of a surface code are in one-to-one correspondence with the homology classes of the lattice and its dual:

$$N(\mathcal{S})/\mathcal{S} \simeq H_1 \times H_1^* \tag{16}$$

2.3 Surface Codes with Boundaries

Embedding a lattice on a closed surface provides a mathematically elegant way to encode quantum information using local interactions. However, the practical feasibility of arranging physical qubits on a closed surface such as a torus is questionable, but arbitrarily truncating a closed surface code to a planar geometry destroys the non-trivial homology classes. Bravyi and Kitaev [4] proposed a solution to this problem by introducing the concept of boundaries in surface codes. By carefully designing the boundaries of the lattice, we can preserve the non-trivial homology classes and hence the logical qubits, while still maintaining local interactions between physical qubits. This leads to the well-known planar surface code, which is one of the most promising candidates for practical quantum

error correction in near-term quantum devices.

This extension of closed surface codes is based on a notion of relative homology [18], as presented in [6]. We consider our 2-dimensional surface with a lattice defined by the graph $G = (V, E, F)$ containing two categories of boundaries: open (or rough) and closed (or smooth). An edge is a boundary if it is incident to a single face of G, and the endpoints of these edges are boundary vertices.

We specify a subset of boundary edges and declare it as open, denoted $\partial_O E$. If a vertex is incident to an edge $E \in \partial_O E$, we call it an open vertex $v \in \partial_O V$. The remainder of boundary edges and vertices are called closed, defining $\partial_C V$ and $\partial_C E$. Physical qubits are allocated exclusively in the interior (non-open) edges, defining the interior sets $\tilde{V} = V \setminus \partial_O V$ and $\tilde{E} = E \setminus \partial_O E$. This means that the vector spaces associated with the chain complex are restricted to the non-open elements, with 0-chains and 1-chains defined as $C_0 = \bigoplus_{v \in \tilde{V}} \mathbb{Z}_2 v$ and $C_1 = \bigoplus_{e \in \tilde{E}} \mathbb{Z}_2 e$, while the 2-chains remain the same $C_2 = \bigoplus_{f \in F} \mathbb{Z}_2 f$.

The boundary operators $\partial_2 : C_2 \to C_1$ and $\partial_1 : C_1 \to C_0$ are defined as in the closed surface but restricting the image of both operators to the interior subsets. The truncation does not alter the nilpotency condition $\partial_1 \circ \partial_2 = 0$. Therefore we can define the group of relative 1-cycles Z_1^∂ as the kernel of ∂_1 and the group of relative 1-boundaries B_1^∂ as the image of ∂_2, and $B_1^\partial \subseteq Z_1^\partial$. By discarding the open vertices in C_0, the definition of a relative cycle now includes subsets of non-open edges with endpoints on open vertices, thus eliminating the necessity of forming a closed path. The first relative homology group is:

$$H_1^\partial = \frac{Z_1^\partial}{B_1^\partial} \tag{17}$$

Intuitively, the introduction of homological degrees of freedom seems obvious, as the boundaries introduce non-trivial cycles represented as open paths on the lattice. To determine the exact number of logical qubits k encoded by a surface code with boundaries, one needs to calculate the dimension of H_1^∂ using the rank-nullity theorem:

$$\dim \ H_1^\partial = \dim Z_1^\partial - \dim B_1^\partial \tag{18}$$

Proposition 2. *The dimension of the first homology group of a generalized surface code is given by:*

$$\dim H_1^\partial = -|\tilde{V}| + |\tilde{E}| - |F| + \kappa_{\overline{\partial_O V}}(G) + \kappa_{\overline{\partial_C E}}(G) \tag{19}$$

Where $\kappa_{\overline{\partial_O V}}(G)$ denote the number of connected components of G containin no open vertices and $\kappa_{\overline{\partial_C E}}(G)$ denote the number of connected components of G containing no closed edges.

Proof. Departing from (18), we first obtain $\dim Z_1^\partial$, the relative cycle space of the open graph $H = (V, E)$. We construct a 2-cover graph H_2 devoid of open vertices by taking two identical copies of H and connecting the corresponding pairs of open vertices with

an extra edge. For this standard graph without open vertices, the dimension is given by $|E| - |V| + \kappa$, where κ is the number of connected components. Considering H_2:

$$\dim Z_1(H_2) = 2|\tilde{E}| + |\partial_O V| - 2|\tilde{V}| + 2\kappa_{\overline{\partial_O V}}(H) + \kappa_{\partial_O V}(H)$$

We define the linear projection $\pi : Z_1(H_2) \to Z_1^\partial$ that maps every cycle in H_2 onto its restriction in the first copy of H. The kernel of π is the set of cycles contained entirely in the second copy of H, and hence:

$$\dim \ker \pi = |\tilde{E}| - |\tilde{V}| + \kappa(\tilde{H})$$

And using the rank-nullity theorem:

$$\dim Z_1^\partial = \dim Z_1(H_2) - \dim \ker \pi = |\tilde{E}| - |\tilde{V}| + \kappa_{\overline{\partial_O V}}(G) \tag{20}$$

Now for the relative boundaries, B_1^∂ is generated by the vectors $\partial_2 f$ for all $f \in F$. If a connected component C of G contains at least one closed boundary edge $e \in \partial_C E$, we cannot find a combination of its faces $c_f \in C_2$, with $c_i \neq 0$ only for $f_i \in C$, such that $\partial_2(c_f) = 0$, so the vectors $\partial_2 f$ associated with faces in this component are linearly independent. If the connected component lacks of any closed boundary edge, summing the boundary operators of all faces in the component yields the zero vector. We then conclude that:

$$\dim B_1^\partial = |F| - \kappa_{\partial_C E}(G) \tag{21}$$

Thus, using equations (18), (20) and (21), we obtain:

$$\dim H_1^\partial = -|\tilde{V}| + |\tilde{E}| - |F| + \kappa_{\overline{\partial_O V}}(G) + \kappa_{\overline{\partial_C E}}(G)$$

□

There are different methods to encode a logical qubit using mixed boundaries, but we are primarily interested in continuous 2-dimensional planar patches completely devoids of internal holes or topological defects. Those are finite planar lattices forming a topological disk.

Consider that we assign a uniform boundary to our planar patch (either closed or open). It is inmediate that

$$\kappa_{\overline{\partial_O V}}(G) + \kappa_{\overline{\partial_C E}}(G) = 1$$

as our lattice is simply connected and it has either open or closed boundaries, but not both. Applying the Euler characteristic for a disk:

$$|V| - |E| + |F| = |\tilde{V}| + |\partial_O V| - |\tilde{E}| - |\partial_O E| + |F| = 1$$

If the entire boundary of the disk is closed, the number of both open vertices and open edges is 0. On the other hand, if the entire boundary is open, the number of vertices and edges in the perimeter are equal and hence $|\partial_O V| - |\partial_O E| = 0$. Therefore:

$$-|\tilde{V}| + |\tilde{E}| - |F| = -1$$

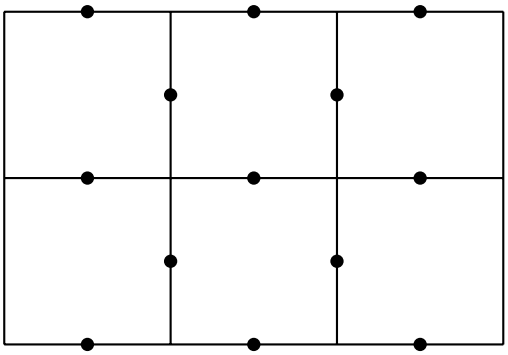

Figure 3: A planar patch. The top and bottom edges are closed (smooth) boundaries and the left and right edges are open (rough) boundaries. The black circles are once again physical qubits, and as we can see the open edges are not associated with any physical qubit.

and using proposition 2:

$$\dim H_1^{\partial} = 0$$

Hence, we conclude the necessity of using mixed boundaries to encode a logical qubit on our planar patch. We specifically propose a rectangular planar surface code where opposing parts of the perimeter are assigned the same type of boundaries, as represented in figure 3.

For any of the sides of the perimeter, $|V_s| - |E_S| = 1$. Therefore, $|\partial_O V| - |\partial_O E| = 2$. Applyin this and the Euler characteristic for the disk:

$$\begin{aligned} -|\tilde{V}| + |\tilde{E}| - |F| &= -|V| + |E| - |F| + |\partial_O V| - |\partial_O E| \\ &= -1 + 2 = 1 \end{aligned}$$

We also notice that $\kappa_{\overline{\partial_O V}}(G) = \kappa_{\overline{\partial_C E}}(G) = 0$. Therefore, the dimension of the first homology group $\dim H_1^{\partial} = 1$, successfully encoding one logical qubit in our planar surface code.

If we consider this prototypical realization of the planar surface code, we see that Z-type star stabilizers at each vertex and X-type plaquette stabilizers at each face in the interior of the lattice simultaneously stabilize four qubits. On the other hand, vertices in the smooth boundaries are only connected to 3 edges, and therefore the Z-type stabilizers along the rough boundaries are weight-3 operators. If we consider faces adjacent to rough boundaries, they are bounded by only three interior edges (therefore three physica qubits), so the X-type plaquette stabilizers along the rough boundaries are weight-3 operators too. Those weight-3 stabilizeres can be seen as algorithmic "sinks", allowing error chains to terminate on the boundaries of the lattice without pushing the system out of the code subspace.

2.4 Logical Operators and the Distance of the Code

Following the connection between homology and logical operators established for the closed surface, we can now define the logical operators for the planar surface code. The

logical Pauli operators correspond to the non-trivial equivalence classes of the relative homology group H_1^{∂} and its dual. A logical X_L operator is represented by a chain of physical X operators along a path connecting two distinct open (rough) boundaries. Such a path is a relative 1-cycle because its boundary endpoints lie entirely within the open vertices, meaning it has an empty boundary in the interior subset $\tilde{V}$. Hence, it commutes with all vertex stabilizers.

Similarly, by considering the dual lattice, a logical Z_L operator is formed by a chain of physical Z operators connecting two closed (smooth) boundaries. These operators commute with all stabilizers but cannot be constructed as a product of them, as they are not relative boundaries. As we showed, a standard planar surface code encodes a single logical qubit ($k = 1$) by featuring two opposite open boundaries and two opposite closed boundaries. This topology allows for one independent X_L path and one independent Z_L path. These paths must intersect at exactly one physical qubit (or an odd number of them), ensuring the necessary anti-commutation relation $\{X_L, Z_L\} = 0$ for the logical Pauli algebra.

The distance of the planar surface code is determined by the minimum weight of a non-trivial logical operator. For the X_L operator, this is the length of the shortest path between the two rough boundaries, often denoted as the horizontal width d_x. For the Z_L operator, the distance is the length of the shortest path between the two smooth boundaries, denoted as the vertical height d_z. The overall distance d of the code is $\min(d_x, d_z)$.

Executing the planar surface code demands continuous syndrome extraction across the lattice. In order to measure the stabilizers, ancillary measure qubits are interlaced among the data qubits [8], as shown in figure 4a.

Syndrome qubits are placed in the centre of each plaquette and in each vertex, and they measure the stabilizer by the application of a standarized sequence of `CNOT` gates, where the data qubits are control and the ancilla qubit is target, thus entangling the ancilla qubit with its surrounding data qubits. The circuits are shown in figures 4c and 4b.

3 Practical Problem

The implementation of quantum algorithms in fault-tolerant arquitectures require their treatment in a model that does not impose the unitary evolution of standard circuit models. The practical problem of this work can be summarized as the compilation of quantum protocols in terms of a problem of topological optimization of tensor networks using ZX calculus.

Given a quantum circuit $\mathcal{C}$, we have a translation map $\mathcal{T} : \mathcal{C} \to D_{ZX}$, where D_{ZX} is a ZX diagram obtained by the substitution of rotations in Z and X bases by green and red spiders, as well as the initialization of qubits and measures [23].

The resource cost of a given computation is naturally captured by its space-time volume [25], defined as its spatial footprint (number of qubits) times its execution time (number of error correction cycles required). As lattice surgery operations exhibit a direct correspondence with spiders in ZX calculus, the topological deformation of a ZX diagram

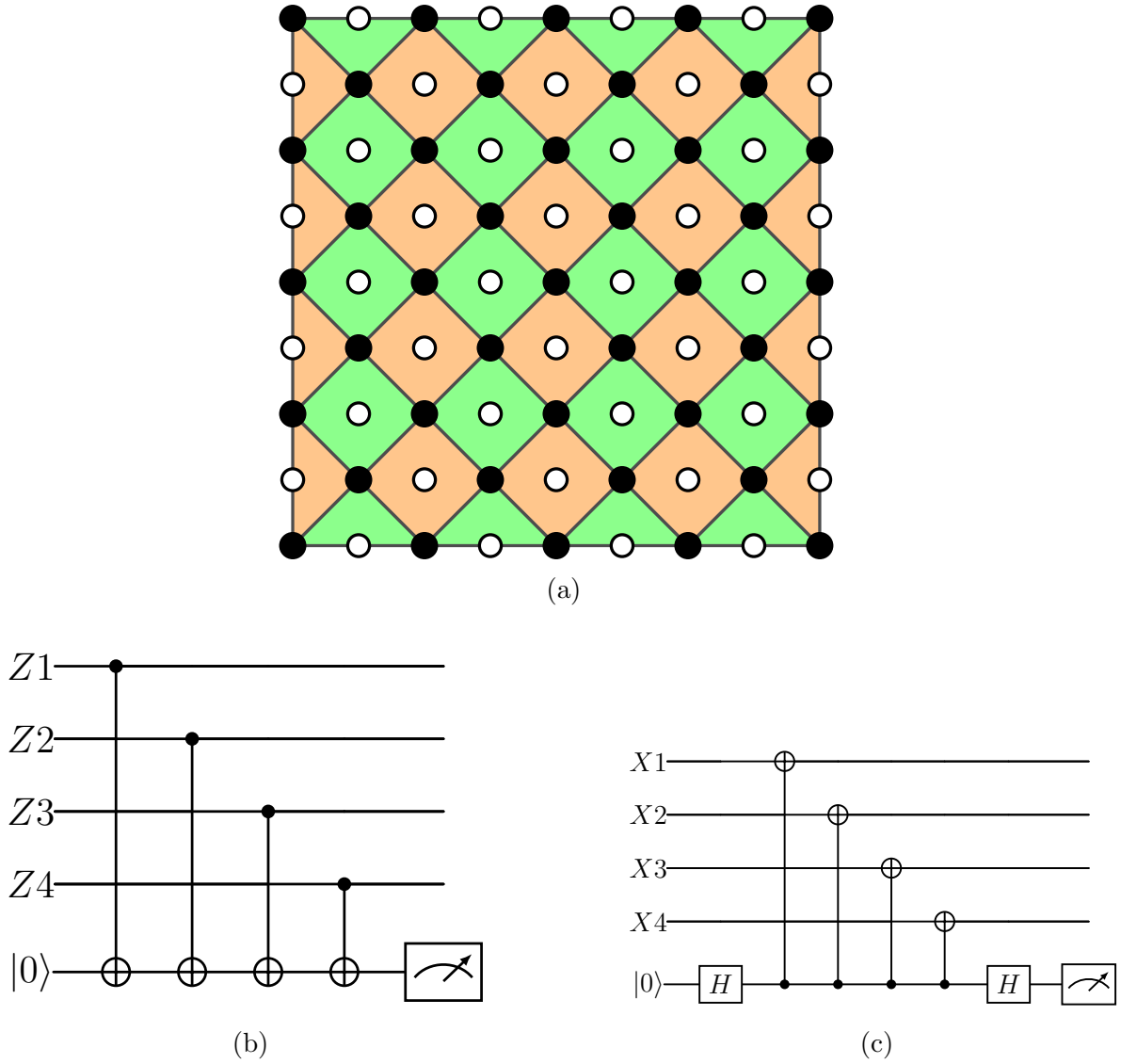

Figure 4: (a) A distance 5 planar surface code. Black dots represent data qubits, and white dots represent ancillary qubits. Green squares are X-type plaquettes stabilizers and orange squares are Z-type star stabilizers. (b) Z-syndrome. (c) X-syndrome.

necessarily implies the reestructuration of its physical operations. Hence, the compilation problem can be formulated as an optimization problem:

$$\min V(D'_{ZX}) \quad \text{such that} \quad [[D'_{ZX}]] = [[D_{ZX}]] \tag{22}$$

where $[[\cdot]]$ represents the lineal map described by the object. Applying rewrite rules allows to drastically optimize the metric. A quantitative evaluation of the space-time volume can be performed on ZX diagrams corresponding to lattice surgery procedures. The number of logical qubits required to implement a sequence of lattice surery operations represented by a ZX diagram is given by the maximum number of wires in any vertical cut of the diagram [23].

For the temporal cost of simple protocols, we can use the layer slicing procedure [25]. We identify the entry nodes of the diagram (usually, the open wires in the left) and they are assigned the layer index $L = 1$. Their topological neighbours are as-

signed $L = 2$. We propagate this process, and each vertex is assignated a layer index $L(v) = \max_{u \in \text{fathers}(v)} L(u) + 1$. The temporal cost of an algorithm can be approximated by the maximum index in the diagram, which we will call the topological depth.

For simple diagrams, the computation of this metric is quite straightforward, but when the protocol requires a high number of quantum gates, the complexity of the diagrams hinders the computation of its topological depth. Multi-qubit gates such as the `CNOT` or `CZ` appear as vertical lines in the ZX diagrams, amd the resolution of this ambiguity in one specific operational instantiation of the gate result in different graphs, hence altering the layer slicing procedure. Different physical realizations for the multi-qubit gates imply different topological depths. The task of reducing the rounds of error correction rounds, and hence of improving the execution time of the algorithm can be seen as an optimization problem in graph theory [22]. We can try to formalize it by giving a more precise definition of topological depth.

Definition 3. *Let $G = (V, E)$ be a directed acyclic graph. We define a set of entry vertices as:*

$$V_1 = \{v \in V | in - degree(v) = 0\}$$

where the in-degree represents the number of edges directed towards v. For any vertex $u \in V \setminus V_1$, we define the depth or the layer index map $L : V \to \mathbb{N}$ in a recursive way:

$$L(u) = \max\{L(v) | (v, u) \in E\} + 1$$

where $(v, u) \in E$ in an edge directed from v to u. We define the topological depth of the graph as:

$$D = \max\{L(v) | v \in V\}$$

When our ZX diagram has differentiated wires representing different logical qubits, we can model it as a mixed graph $G = (V, E_H, E_V)$, where E_H are the horizontal edges along a wire and E_V are the vertical edges for each multi-qubit gate. The fact that the vertical edges admit different physical realizations can be translated to this set up by considering E_H as a set of directed edges such that they conform separate directed, non-intersecting, open paths (wires), and E_V as a set of non-directed edges connecting vertices in different horizontal paths. The direction of the edges can be associated with a time-like direction, and the first vertex of each wire represents an input of the diagram, and the last vertices correspond to the outputs. The specific physical realization of the gate is an assignation of an orientation to its corresponding vertical edges. Given a mixed graph G, an orientation $\mathcal{O}$ is a function that assigns a direction to each $e \in E_V$. For an orientation $\mathcal{O}$ to be valid, the resulting graph $G^{\mathcal{O}} = (V, E_H \cup E_H^{\mathcal{O}})$ must be acyclic.

Furthermore, the physical impositions of the hardware allow us to restrict the graph model further. The graph contains a finite number of wires $\{W\}_i$. Given any wire W_1, three vertices $u_1, u_2 \in W$ and two edges $(u_1, v_1), (u_2, v_2) \in E_V$ such that $v_1 \in W_2$, $v_2 \in W_3$ and $W_2 \neq W_3$, then if any other vertex $u_3 \in W_1$ has a vertical edge attached $(u_3, v_3) \in E_V$, then v_3 is either in W_2 or W_3. The restriction introduces the concept of neighbour wires, i.e., wires in the graph that may be linked with vertical edges. Other conditions can be defined for different connectivity restrictions. This model is represented in figure 5. The

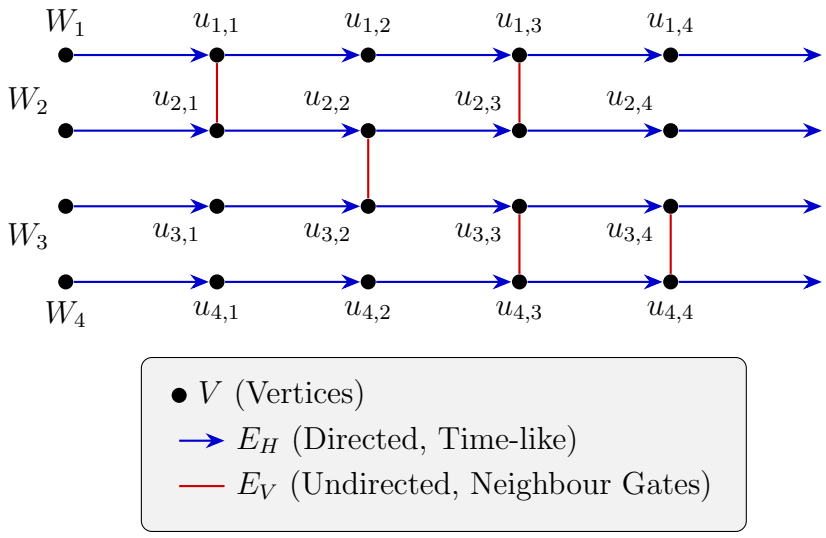

Figure 5

depth of a vertex v can be interpreted as the longest directed path starting at any input vertex and ending at v.

Given this model for our diagrams, the time-optimization problem in a graph G is solved by searching the orientation $\mathcal{O}$ that yields the minimum topological depth $D(G^{\mathcal{O}})$. It is clear that $D(G^{\mathcal{O}}) \geq D(G)$, as the length of the directed paths cannot decrease after we apply an orientation.

In order to construct an algorithmic solution for the problem, let's consider how an specific iteration could be performed: we start reading the graph from the input vertices (in figure 5, from the left to the right) until we find the first vertex attached to a vertical edge e. The two vertices connected by this edge, v and u have a primitive layer index $L_{in}(u)$ and $L_{in}(v)$ respectively, determined by their preceeding horizontal edges (and, for later iterations, by their preceeding directed vertical edges too). If we direct e from $u \to v$, the new depth of v will be $L_{out}(v) = \max\{L_{in}(v), L_{in}(u) + 1\}$, and $L_{out}(u) = L_{in}(u)$. It is inmediate that, if $L_inv > L_{in}(u)$, then $L_outv = L_{in}(u)$. Therefore, directing the edge towards the vertex with the highest depth does not produce any change on the depth of the vertices, and hence, on the total topological depth of the graph.

If we have a vertical edge connecting two vertices (u, v) with $L_{in}(v) = L_{in}(u)$, or strictly vertical edges, we need to define a new metric $S : V \to \mathbb{N}$, which we will call the shallowness of the vertex, and it will be given by the length of the horizontal path connecting the vertex in the argument and the output vertex of the wire. For an edge (u, v) with $L_{in}(v) = L_{in}(u) = k$ and, without loss of generality, $S(u) < S(v)$, let's consider the local impact of the orientation on the cost of their respective paths $L_{out}(v) + S(v)$ and $L_{out}(u) + S(u)$:

- $u \to v$: $L_{out}(u) = k$, and the total cost of its path will be $k + S(u)$. $L_{out}(v) = k + 1$ and the total cost will be $k + 1 + S(v)$. The maximum local cost is $\max\{k + S(u), k + 1 + S(v)\} = k + 1 + S(v)$
- $v \to u$: $L_{out}(v) = k$, and the total cost of its path will be $k + S(v)$. $L_{out}(u) = k + 1$

and the total cost will be $k+1+S(u)$. The maximum local cost is $\max\{k+S(v), k+1+S(u)\} = k + S(v)$

Hence, the orientation of the vertical edges should be towards the shallowest vertex. If we already assigned an orientation to every edge between vertices with different in-path depth (those do not produce any change in the depth, neither locally nor globally), we can take a global approach for a systematic treatment of the vertical edges. We have already proved that the orientation of those vertical edges produces an increase in the total length of one the wires. If we have several vertical edges connecting the two wires in vertices with the same in-path depth, once we direct the first one from the left, the remaining edges will have a trivial orientation, as the depth of the vertices in one of the wires will increase by one. This means that if we direct all those vertical edges towards the same wire, the depth of the vertices in that wire will only increase by one. For instance, in figure 6, if we direct (u_1, v_1) and (u_2, v_2) upwards, $L_{out}(u_3) = L_{in}(u_3) + 1$. If a given wire W has those edges (between vertices with the same in-path depth) with two different wires, and we direct all of them towards W, the total depth of the wire will also increase by 1.

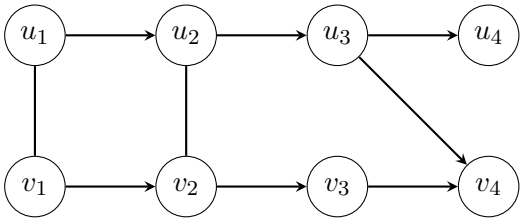

Figure 6

In cases where there are subsequent vertical edges connecting vertices u_i and v_j such that $L_{in}(v_j) - L_{in}(u_i) = 1$, (in figure 6, (u_3, v_4)) either orientation for the vertical edges will increase the depth of v_j, while if the vertical edges are directed towards the wire that contains v_j, then $L_{in}(u_i) = L_{out}(u_i)$. Therefore, in those situations the vertical edges should be directed towards the wire containing v_j.

If this case does not hold, a decision must be made, as one of the wires will necessarily suffer an increase in its depth. As we try to minimize the global depth of the graph, we make this decision by looking at the total length of all the wires. Firstly, we find the longest wire in the diagram, and direct all the strictly vertical edges attached to it towards its neighbours. This way we ensure that the total depth of the diagram does not increase. The remaining strictly vertical vertices will be oriented in an alternate way: if a wire already has strictly vertical edges oriented towards it (as in the case of the neighbours of the longest wire), the remaining strictly vertical edges attached to it will be also oriented towards it. The same thing applies for wires with directed vertical edges towards its neighbours. The criteria can be visualized in figure 7. If there are two wires with maximum length separated by an odd number of wires, this method does not present any conflict. If they are separated by an even number of wires, any of those intermediate wires would increase its total depth by 2, so the total depth of the diagram could increase by 1. Considering the case discussed above, that could also increase the lenght of the longest wire by 1, this method provides an orientation $\mathcal{O}$ such that the topological depth of the graph $D(G^{\mathcal{O}}) \leq D(G) + 2$.

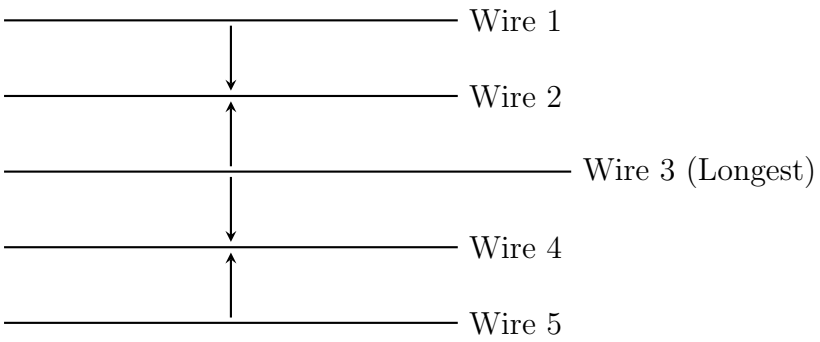

Figure 7

The algorithm would be as follows:

- We assign the trivial orientations for the vertical edges connecting two vertices with different in-path depth.
- Among those oriented vertical edges, we search for edges (u_i, v_j) satisfying $L_{in}(v_j) - L_{in}(u_i) = 1$. If there are previous strictly vertical edges connecting the same two wires, those are directed towards the wire containing v_j
- We identify the wires with the greatest length, and apply the assignation represented in 7

For the computational implementation of the problem, we will use PyZX framework, an open source python-based library specifically designed for diagramatic reasoning in ZX calculus [12]. At the software architecture level, PyZX abstract the problem usin two main data-structures: circuits and graphs.

The circuit class captures the standard representation of quantum protocols as a sequential application of quantum gates. On the other hand, the graph class instantiates the equivalent ZX diagram, which provides a more interesting object for our purpose.

This library has built-in hierarchical simplification strategies. The rules consist of a matcher and a rewriter. At a lower level, the matchers is nothing but a composition of heuristic routines searching for subgraphs satisfying the structural conditions for any of the axiomatic rules of ZX calculus. Once it returns the list of matches, the rewriter applies the corresponding simplifications. [AMPLIAR CON REWRITE STRATEGIES COMPLETAS].

This framework will provide the necessary tools for the algorithmic implementation of quantum routines in ZX diagrams, as well as for their topological optimization.

3.1 Logical Verification and Physical Synthesis Duality

PyZX has certain routines, such as `full_reduce`, that simplify the original diagram in a way that could break its physical interpretation in terms of lattice surgery operations. This makes evident the two computational regimes in which ZX diagrams may operate, at least in the context of this work.

On the one hand, ZX calculus represents an intermediate representation of hardware operations where each transformation saves physical resources [25]. For instance, the spiders fusion rule reflects the physical fact that two sequential split (or merge) operations along the same type of boundaries can be unified. This approach helps to reduce the space-time volume of a given protocol.

On the other hand, ZX calculus can be used for logical verification of semantic equivalence. Complex sets of unitary gates can be transformed in a ZX diagram, and its simplification can manifest semantic equivalence with a more intuitive process. This will be clearly illustrated in the implementation of quantum teleportation.

3.2 Quantum Teleportation

Quantum teleportation is a well-known protocol that transfers an unknown quantum state from one qubit to another [19]. In order to perform this, the protocol consumes two resources: pre-shared quantum entanglement and classical information communication. The standard realization of quantum teleportation is depicted in figure 8.

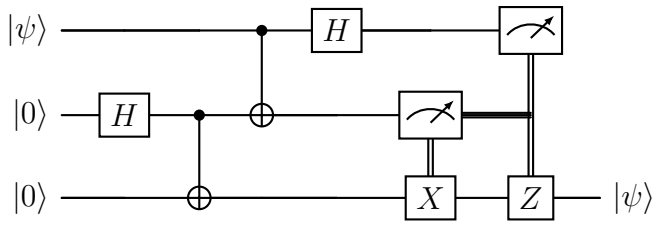

Figure 8: Stardard quantum teleportation circuit.

Consider Alice has a qubit in an initial quantum state $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$. Alice and Bob share a maximally entangled pair of qubit, for instance

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

so the initial state is:

$$|\psi_0\rangle = |\psi\rangle \otimes |\Psi^+\rangle = \frac{1}{\sqrt{2}}[\alpha(|000\rangle] + |001\rangle) + \beta(|100\rangle + |111\rangle)]$$

Alice performs a local interaction with the first two qubits. Firstly, she applies a `CNOT`, using the unknown qubit as control, and then she applies a Hadamard gate on it. The transmitter's qubits are mapped to a Bell state. The lobal state is given now by:

$$\begin{aligned}|\psi_1\rangle = \frac{1}{2}[&|00\rangle\,(\alpha\,|0\rangle + \beta\,|1\rangle) + |01\rangle\,(\alpha\,|1\rangle + \beta\,|0\rangle) \\ &+ |10\rangle\,(\alpha\,|0\rangle - \beta\,|1\rangle) + |11\rangle\,(\alpha\,|1\rangle - \beta\,|0\rangle)]\end{aligned}$$

Alice then performs a proyective measure on her two qubits, obtaining two classical variables $m_1, m_2 \in \{0, 1\}$. This measure destroys local entanglement, but transports the original unknown state to Bob's qubit. It actually transmits an encrypted version of $|\psi\rangle$, depending on the classical variables:

- If $(m_1, m_2) = (0, 0)$, the state is $\alpha\,|0\rangle + \beta\,|1\rangle$ $(|\psi\rangle)$.
- If $(m_1, m_2) = (0, 1)$, the state is $\alpha\,|1\rangle + \beta\,|0\rangle$ $(X\,|\psi\rangle)$.
- If $(m_1, m_2) = (1, 0)$, the state is $\alpha\,|0\rangle - \beta\,|1\rangle$ $(Z\,|\psi\rangle)$.
- If $(m_1, m_2) = (1, 1)$, the state is $\alpha\,|1\rangle - \beta\,|0\rangle$ $(ZX\,|\psi\rangle)$.

To recover the original state, we need to transmit the two classical bits to Bob and perform the corresponding Pauli gate. The classical control is useful because it allows us to realize the teleportation protocol without violating connectivity restrictions. However, as PyZX does not have tools to translate classical operations to ZX diagramas, we apply the deferred measurement principle [19] to represent the classical control in terms of quantum gates, as shown in figure 9.

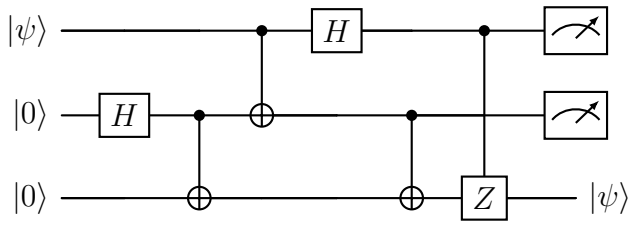

Figure 9: Stardard quantum teleportation circuit without classical control.

The election of quantum teleportation as the first routine to implement and optimize does not respond to an arbitrary volition, but rather to its structural relevance due to hardware limitations. The restricted local connectivity of fault-tolerant arquitectures makes the implementation of logical gates among distant planar patches prohibitive in terms of resources. Therefore, teleportation acquires an undisputed importance, allowing the propagation of quantum information along the processors, as well as being a fundamental ingredient for the transversal implementation of non-Clifford gates [2], surpassing the technical limitations imposed by the Eastin-Knill theorem.

3.3 CHSH Game

The interest of implementing CHSH game (named after Clauser, Horne, Shimony and Holt [5]) resides on two reasons: it provides sufficient proof to demonstrate quantum advantage against any hidden variables theory; and it also justifies the introduction of non-Clifford resources in fault-tolerant architectures.

CHSH game is a problem in information theory that can be modeled as a non-local cooperative game [24]. Two distant players, Alice and Bob, receive each a classical bit of information $x \in \{0, 1\}$ and $y \in \{0, 1\}$. Without any communication, each of them must produce an output bit $a \in \{0, 1\}$ and $b \in \{0, 1\}$, such that they satisfy this boolean relation:

$$a \oplus b = x \cdot y$$

For any strategy restricted to classical phenomena, the probability of winning the game is upper bounded $P_{classic} \leq 0.75$. We can define a quantum strategy that violates this limit. This requires that Alice and Bob share an entangled pair of quibts, characterized by a Bell state $|\Psi^+\rangle = 1/\sqrt{2}(|00\rangle + |11\rangle)$. Let's discuss the quantum strategy in detail. We start by defining a qubit state vector $|\psi_\theta\rangle$ in terms of an angle θ:

$$|\psi_\theta\rangle = \cos(\theta)\,|0\rangle + \sin(\theta)\,|1\rangle$$

The inner product of any two of these vectors is given by:

$$\langle\psi_\alpha|\,|\psi_\beta\rangle = \cos(\alpha)\cos(\beta) + \sin(\alpha)\sin(\beta) = \cos(\alpha - \beta)$$

and if we commpute the inner product of the tensor product of any of these vectors with the Bell state $|\Phi^+\rangle$:

$$\langle\psi_\alpha \otimes \psi_\beta|\,|\Phi^+\rangle = \frac{\cos(\alpha - \beta)}{\sqrt{2}} \tag{23}$$

We also define a unitary matrix U_θ:

$$U_\theta = |0\rangle\langle\psi_\theta| + |1\rangle\langle\psi_{\theta+\pi/2}|$$

Which in a standar convention would be $U_\theta = R_y(-2\theta)$. Alice and Bob share an entangled pair of qubits (A,B) in state $|\Phi^+\rangle$. They receive the physical bit of information, and then perform an operation on their qubits, depending on which classical variable they received. If Alice receives $x = 0$, she will perform U_0, and $U_{\pi/4}$ for $x = 1$. On the other hand, Bob will perform $U_{\pi/8}$ for $y = 0$ and $U_{-\pi/8}$ for $y = 1$. After they perform this operation, they make a measure of their qubits, and the classical output they obtain are their answers.

For illustrative purposes, let's consider the case $(x, y) = (0, 0)$ using equation 23:

$$\begin{aligned}(U_0 \otimes U_{\pi/8})\,|\phi^+\rangle &= |00\rangle\langle\psi_0 \otimes \psi_{\pi/8}|\,|\phi^+\rangle + |01\rangle\langle\psi_0 \otimes \psi_{5\pi/8}|\,|\phi^+\rangle \\ &\quad + |10\rangle\langle\psi_{\pi/2} \otimes \psi_{\pi/8}|\,|\phi^+\rangle + |11\rangle\langle\psi_{\pi/2} \otimes \psi_{5\pi/8}|\,|\phi^+\rangle \\ &= \frac{\cos\left(-\frac{\pi}{8}\right)|00\rangle + \cos\left(-\frac{5\pi}{8}\right)|01\rangle + \cos\left(\frac{3\pi}{8}\right)|10\rangle + \cos\left(-\frac{\pi}{8}\right)|11\rangle}{\sqrt{2}}.\end{aligned}$$

And the probability of winning the game is:

$$P(a = b) = \cos^2(-\frac{\pi}{8}) = \frac{2 + \sqrt{2}}{4} \approx 0.85$$

For the remaining three cases, the probability is exactly the same [24], violating the classical limit of $P_{classic} \leq 0.75$. The realization of this strategy in standard circuit model can be seen in figure 10.

Alice's rotations U_0 and $U_{\pi/4}$ are the identity operation and a Hadamard gate. On the other hand, Bob's rotations $U_{\pi/8}$ and $U_{-\pi/8}$ introduce non-Clifford operations. This enforces the use of magic-state distillation, as the Eastin-Knill theorem prevents us from obtaining a transversal realization of the T gate on a planar surface code. But, as we have already discussed, the injection of magic-states is extremely costly.

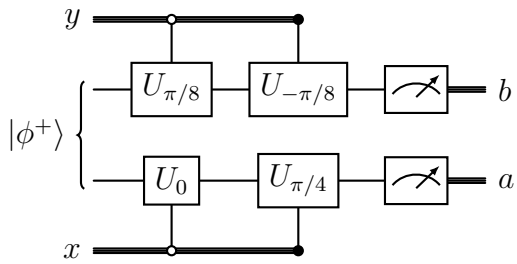

Figure 10: CHSH circuit for $(x, y) = (0, 0)$

When we convert our circuit model to a ZX diagram, PyZX will model the T gate as a phase gadget. This allows us to push the non-Clifford gates through the diagram to find non-local cancelations [13], reducing the T-count and hence the computational cost of the circuit.

3.4 Magic Square Game

The Mermin-Peres magic square game is a pseudo-telepathy scenario that illustrates quantum contextuality [17]. It is once again a cooperative non-local game played by two distant players, Alice and Bob. They are given a 3 x 3 square which they must fill in a specific way. Alice is asigned a random row, and Bob is given a random column. Each must complete their corresponding cells with values ± 1, satisfying the following conditions:

- Alice's row must contain an odd number of plus signs.
- Bob's column must contain an odd number of minus signs.
- The cell where Alice's row and Bob's column intersect must contain the same value for both players.

It is easy to see that no classical strategy can produce a victory probability of 100%, as this would imply the construction of a 3 x 3 matrix where all the rows add up to +1 and all the columns add up to -1, so the global product of the matrix evaluated on the rows would differ from that evalauted on the columns.

However, the use of quantum resources may ensure a deterministically winning strategy. Alice and Bob would need to share two pairs of maximally entangled pairs of qubits. For instance, we could have a global state could described by $|\Phi^+\rangle \otimes |\Phi^+\rangle$. Each of them will take one qubit from each pair.

Instead of buildin a 3 x 3 matrix with ± 1 values, we construct a 3 x 3 ggrid composed of elements of $\mathcal{P}_2$. We can use this specific arrangement:

$+I \otimes Z$	$+Z \otimes I$	$+Z \otimes Z$
$+X \otimes I$	$+I \otimes X$	$+X \otimes X$
$-X \otimes Z$	$-Z \otimes X$	$+Y \otimes Y$

Now the referee gives them their assignments, and they perform the corresponding measurements on their qubits based on the grid above. As we can see, the three operators in any of the rows commute, so Alice can measure them simultaneously. If you multiply the three operators in any row, they yield $I \otimes I$. The product of Alice's three measurement will always be $+1$. On the other hand, the operators in any of the columns also commute, so Bob can measure simultaneously too. The product of the three operators in any column is $-I \otimes I$, so the product of Bob's measurements will always be -1.

As the qubits are in a Bell state, when Alice and Bob make the measures corresponding to the same cell in the grid, they will obtain the same result. The three conditions are fulfilled, and hence we have defined a deterministically winning quantum strategy.

The implementation of this game in the unitary circuits paradigm begins by preparing the two pairs of entangled qubits. Then, a classical control decides the specific unitary rotations that Alice and Bob apply on their qubits. To illustrate how these measures are performed, let's consider the case where Alice is assigned the third row and Bob gets the first column. We consider that measurements can be performed in the Z direction, and hence we need to make some necessary rotations to measure the required observables. Alice's first two observables are $-X \otimes Z$ and $-Z \otimes X$. We want to understand the necessary transformation of these operators in the Heisenberg picture in order to obtain the measurable observables $Z \otimes I$ and $I \otimes Z$. Specifically, we want to find a unitary sequence U that maps Alice's two first observables into single-qubit Z measurements:

- $-X \otimes Z \to Z \otimes I$
- $-Z \otimes X \to I \otimes Z$

We can clearly see how `CZ` gate can disentangle the operators:

$$CZ(X \otimes Z)CZ = X \otimes I$$
$$CZ(Z \otimes X)CZ = I \otimes X$$

Now we rotate to the Z-axis:

$$(H \otimes H)(-X \otimes I)(H \otimes H) = -Z \otimes I$$
$$(H \otimes H)(-I \otimes X)(H \otimes H) = -I \otimes Z$$

We finally fix the parity by applying X gates:

$$(X \otimes X)(-Z \otimes I)(X \otimes X) = Z \otimes I$$
$$(X \otimes X)(-I \otimes Z)(X \otimes X) = I \otimes Z$$

Hence, by applying this specific sequence of operators and measuring the first two qubits, we obtain the eigenvalues for the observables $-X \otimes Z$ and $-Z \otimes X$. We also notice that the last observable can be expressed in terms of the first two, $(Y \otimes Y) = (-X \otimes Z) \cdot (-Z \otimes X)$. The product of the three observables of any given row is always $I \otimes I$, and for any column $-I \otimes I$. So Alice's answer for cell 3 can be determined by obtaining the product of the first two eigenvalues, and equivalently for Bob. The specific implementation of this game can be seen in figure 11.

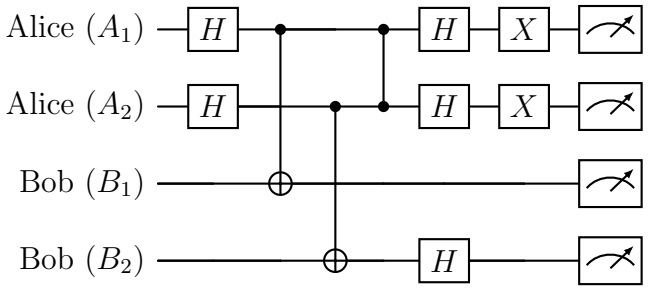

Figure 11: Mermin-Peres game circuit implementation for the first column and the third row.

4 Results

We discuss the PyZX implementations of the three quantum routines presented above.

4.1 Teleportation in PyZX

In order to implement quantum teleportation in PyZX, we need to represent the protocol using the operational description of standard circuit model, as in figure 9. Once we instantiate that circuit and translate it to a graph, the resulting diagram will model the realisation of the protocol with lattice surgery operations, considering the necessary classical operations to update the Pauli frame.

Firstly, we instantiate the logical gates of the circuit and convert it to a graph. This conversion does not include neither the preparation of the initial states nor the final measures. For the EPR pair, we need to initialize both logical qubits in the base state $|0\rangle$, which is performed by adding a red degree-1 node at the begining of each line. Alice's projective measures are performed in an identical way. Finally, the realization of the classical control admits different options:

- **Partial Quantum Assimilation** We simply instantiate the circuit in figure 9. The resulting diagram will have operations among separate qubits, apparently violating the 2DNN connectivity. However, we do not interpret those edges as quantum interactions, but rather as the diagramatic representation of the classical control after the projective measures. This approach allows the simplification routines to absorb the measures subproducts, while also ensuring the semantic equivalence between the compiled graph and the original circuit.

- **Strict Quantum Assimilation** We could consider both `CNOT` and `CZ` gates as genuine quantum interactions. In order to preserve 2DNN connectivity, this approach forces us to include a `SWAP` gate in the first two qubits. The computational cost of this gate makes this option ignorable, as the interest of the teleportation protocol consists precisely in the possibility of replacing the necessity of `SWAP` gates.
- **Classical Control Externalization** We supress the classical control operations in our diagram, faithfully representing the quantum operations involved in the protocol. This option breaks the possibility of performing semantic verification.

If we follow the first approach, we should translate the effects of the projective measures using red degree-1 nodes representing the covectors corresponding with each of the possible results. The deterministic branch where Alice measures both of her qubits in state $|0\rangle$ is shown in figure 12. By applying a π phase in the projective nodes, we could represent results in the state $|1\rangle$.

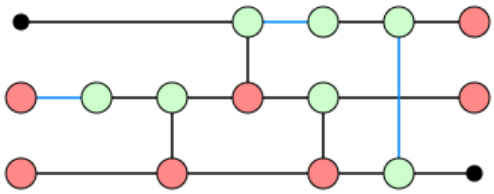

Figure 12: ZX diagram corresponding to a standard quantum teleportation when Alice measures $(0, 0)$.

For each of the diagrams $D_{ZX}(m_0, m_1)$, $m_0, m_2 \in \{0, 1\}$, we perform the simplification routine of `full_reduce`. The result is that the four diagrams collapse to the identity channel $\mathcal{I}$, as shown in figure 13. This constitutes the validation of the implemented protocol.

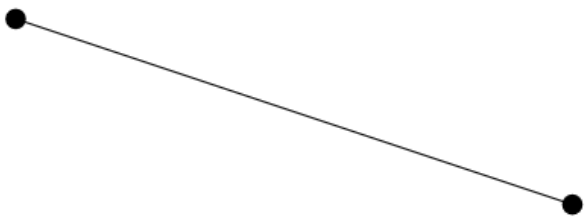

Figure 13: Identity channel resulting from the simplification of the teleportation diagrams

The diagram in figure 12, ignoring the interactions modeling the feedforward, can be viewed in terms of lattice surgery operations (given that the Pauli frame is correctly updated). We can break the simplification routine in each of its steps, and look for simplifications of the protocol that do not break the physical interpretation of the diagram as composition of primitives of lattice surgery. The four diagrams yield the same result prior to the total collapse to the identity channel, as shown in figure 14.

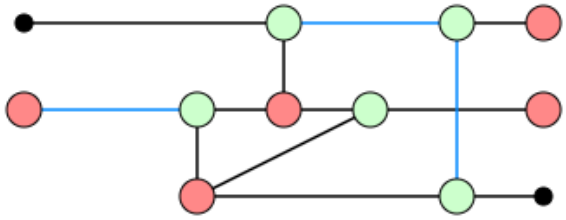

Figure 14: Simplification of the teleportation diagram when Alice measures $(0, 0)$.

The diagram has a reduced number of degree-3 nodes, but we notice that the simplification comes from the topological assimilation of part of the classical control, specifically the controlled X operation, contracting it with the previous quantum `CNOT` operation. Therefore, the space-time volume of the protocol is not reduced.

In order to compile the protocol we try by externalizing the classical control, exclusively considering the quantum operations. The corresponding diagram is shown in figure 15

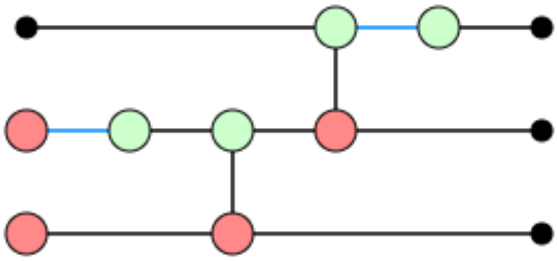

Figure 15: Teleportation protocol without the classical control operations.

Once again, instead of using the `full_reduce` routine, we break it into each of its simplification steps. In this case, the complete routine yields an interesting diagram:

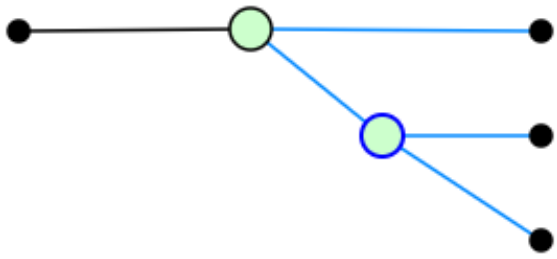

Figure 16: Simplified teleportation protocol without the classical control operations.

Previously, the classical control operations allowed the simplification routine to collapse the diagram to the logical identity. Now, the result of the routine can be put in a direct isomorphism with lattice surgery operations. We have a unique entry node representing the original state $|\psi\rangle_L$ encoded in a logical qubit conformed by three logical

patches, and through two smooth splits we obtain three highly entangled logical qubits. By performing the projective measures on the two first qubits, and the corresponding Pauli operations on the third, we obtain the origianl state $|\psi\rangle_L$ in the last qubit.

If we compare figures 15 and 16, we see that the maximum layer index is reduced from 4 to 3. This represents a huge improvement in the execution time of the algorithm. Also, the pathwidth is reduced from 4 to 3, optimizing the number of logical patches necessary for the implementation.

The physical interpretation presents a problem: the logical patch where we want to teleport the unknown state is originally merged in the logical qubit where that unknown state is encoded. If we want to separate the original state and the target qubit, we can find another implementation in a previous step of the simplification routine:

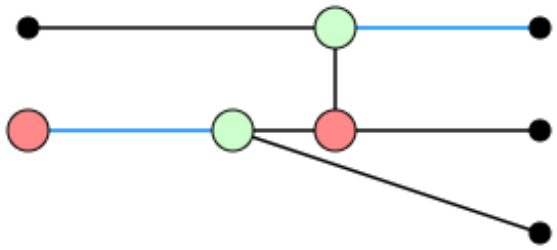

Figure 17

Now, the original state represents an independent entry wire, while the two entangled logical qubits are firstly merged and prepared in the $|0\rangle_L$ state and then separated through a smooth split. The first two qubits are then entangled. This realization of the algorithm reduces the maximum layer index from 4 to 3, and the pathwidth is not reduced. It represents a smaller optimization, but a more useful realization.

4.2 CHSH Game in PyZX

In contrast to the evaluation of the teleportation protocol—where the compiler's objective resided in preserving open logical wires to demonstrate the invariance of the identity channel $\mathcal{I}$—the topological instantiation of the CHSH game is formulated as the strict evaluation of a closed tensor network.

In the diagrammatic formulation using PyZX, the classical variables of the referee ($x, y \in \{0, 1\}$) modulate the local rotation sequences, while the joint measurement outcomes ($m_0, m_1 \in \{0, 1\}$) are imposed as strict boundary conditions on the output ports. These are instantiated via arity-1 tensors of type X (red spiders) with phases $m_i\pi$. This total closure of the topological boundaries, as shown in figure 18, transforms the diagram from a linear matrix operator $\mathcal{H} \to \mathcal{H}$ into a rank-0 tensor, i.e., a complex scalar. Semantically, the closed graph represents the geometric isomorphism of the exact probability amplitude for each branch:

$$\mathcal{A}_{x,y}(m_0, m_1) = |m_0 m_1\rangle \, U_{x,y} \, |00\rangle \tag{24}$$

By iterating programmatically over the 16 possible permutations dictated by the combinations of (x, y) and (m_0, m_1), and subjecting each resulting diagram to the algebraic

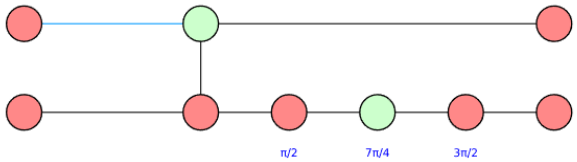

Figure 18: ZX diagram corresponding to $(x, y) = (0, 0)$ and $(m_0, m_1) = (0, 0)$

simplification heuristics, a singular analytical phenomenon is observed: the tensor network collapses entirely into a visually empty diagram.

Far from representing an algorithmic error or a loss of information, the topological annihilation of all nodes and edges constitutes the geometric manifestation of a successful **strong simulation**. The rewriting engine of the Frobenius subalgebra contracts the entirety of the lattice surgery isometries alongside the non-Clifford rotations ($\pm\pi/4$), fully resolving the underlying system of algebraic equations. The empty diagram signifies that the ZX-calculus has successfully evaluated the full trace of the tensor network, encapsulating the complex amplitude $\mathcal{A}_{x,y}(m_0, m_1)$ within the global scalar factor of the remaining graph.

This absolute collapse validates the compiler's capability to act as a deterministic evaluator of multipartite systems. By mathematically recovering the scalar hidden within each "empty diagram", one obtains direct access to the exact marginal probability distribution $P(m_0, m_1|x, y) = |\mathcal{A}_{x,y}(m_0, m_1)|^2$ [14] [21]. This extraction is a fundamental prerequisite for the subsequent certification of the analytical violation of the local hidden-variable (LHV) bounds within the surface code topology.

Indeed, once we extract the marginal probability amplitude from each of the diagrams and compute the probability of victory for each pair of classical variables (x, y), we obtain $P_{quantum} \approx 0.85$, violating the LHV bounds.

Now we can connect a key element of quantum information theory with the hardware realities of fault-tolerant architectures by discussing the implementation and compilation of the protocol. As we did in the teleportation diagram, we exclude the measurement projections:

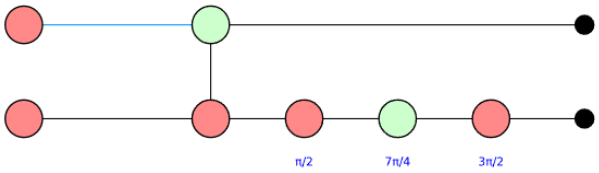

Figure 19: ZX diagram for the implementation of CHSH game, corresponding to $(x, y) = (0, 0)$.

Bob's measurements require rotations in the Y axis with angles of $\pm\pi/4$, which are not elements of the Clifford group. The simplification routines cannot annihilate them, and this is not an avoidable hardware limitation but rather a consequence of the Eastin-Knill theorem, as our fault-tolerant architecture necessarily lacks of a universal set of transversal quantum gates.

Consequently, the realization of the CHSH protocol forces the compiler to invoke magic state injection protocols [20], which are extremely costly, as we have already discussed. The greatest contribution to the space-time volume for this realization comes from the requirement of non-Clifford resources, not from generating the initial entanglement.

Even so, we perform our simplification routine to each of the four possible diagrams. The procedure yields a significat reduction of Clifford gates, which, as we have already stated, does not imply a huge reduction of execution time, but it is still an optimization. The topological depth for each of the four branches is reduced from 5 to 3. The pathwidth

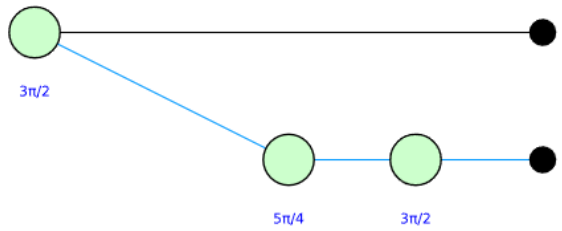

Figure 20: Simplification of the ZX diagram for the implementation of CHSH game, corresponding to $(x, y) = (0, 0)$.

remains the same. The traslation of the diagram to lattice surgery primitives is inmediate:

- We prepare a logical qubit in the state $|0\rangle + e^{i3\pi/2} |1\rangle$.
- A smooth split is performed, and then we apply a T gate on Bob's qubit.
- A final S gate is performed on Bob's qubit, and after that we measure each qubit.

The realization in figure 20 could not be feasible due to the fact that we are preparing an initial state that does not belong to the computational basis. Another realization with a better physical translation is shown in 21

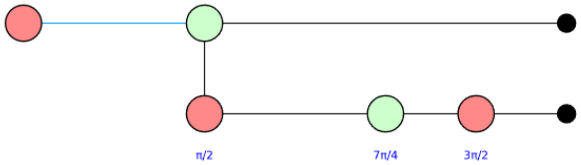

Figure 21: Simplification of the ZX diagram for the implementation of CHSH game, corresponding to $(x, y) = (0, 0)$, and with preparation of initial state in the computational basis.

The topological depth is reduced from 5 in figure 19 to 4 in figure 21.

4.3 Magic Square Game in PyZX

The evaluation of the Mermin-Peres magic square game introduces the verification of quantum contextuality and pseudo-telepathy in fault-tolerant architectures. Unlice the CHSH protocol, which pursues a statistical limit, the Mermin-Peres game demands the

verification of a strict deterministic victory, overcoming the classical bound of 8/9 for this problem.

The protocol requires Alice and Bob to share two pairs of entangled qubits. The instantiation of the circuit begins by creating those two pairs. Next, each player performs the operations corresponding to their observables for the determined row and column. This produces nine different diagrams, with three different subdiagrams for each player's qubit.

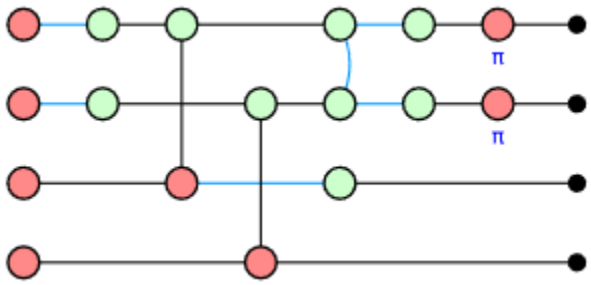

Figure 22: ZX diagram corresponding to the circuit in figure 11.

The validation of the strategy requires a similar approach to the CHSH game. We firstly notice that, once we set boundary conditions in the output wires, placing red degree-1 nodes corresponding to measurement projections, as shown in 23, the simplification routine `full_reduce` annihilates the tensor network, collapsing towards an empty graph.

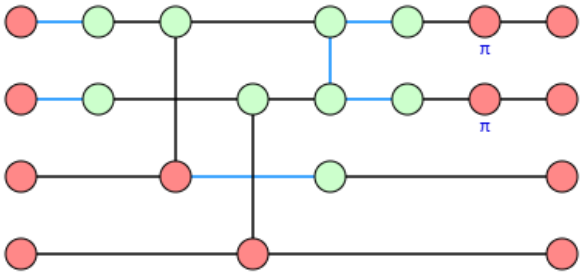

Figure 23: ZX diagram for row 3 and column 0 with boundary conditions corresponding to the covector $\langle 0000|$.

As we saw in the CHSH game, this is an expected consequence from the fact that the graph does not have open wires, meaning it represents a rank-0 tensor (a scalar). The collapse allows us to evaluate the protocol, as the probability amplitude of each stochastic branch is encapsulated withi the global scalar factor of the diagram.

Hence, we extract those amplitudes and obtain the output probabilities for each selection of row and column. By comparing those outputs with the winning conditions, we are able to compute the success probability of the strategy. The application of this method yields us a probability of victory of 1.0, verifying the deterministically winning strategy.

The translation of the Mermin-Peres game, from its abstraction in quantum information theory towards its realization in a fault-tolerant architecture,requires some modifications to the previous diagrams, attending to the spatial limitations imposed by the

hardware.

The initial preparation of the Bell pairs assume global connectivity, which is manifested in the first two `CNOT` gates of figure 22. The physical execution of those gates is not viable under the hardware restrictions we are considering for our architecture. In order to fix this and preserve 2DNN connectivity, we are forced to use a `SWAP` gate, which constitutes an importat operative bottleneck. The realization of the protocol will then begin by the preparation of two Bell pairs, and then we will perform a `SWAP` gate to interchange one qubit from each pair.

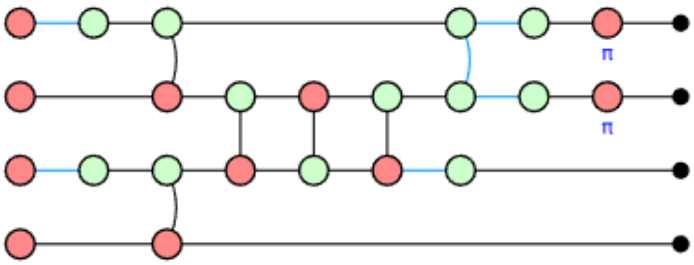

Figure 24: ZX diagram for row 3 and column 0 with 2DNN connectivity.

Compared to the implementation of the CHSHS game, the Mermin-Peres games does not require the use of non-Clifford gates, thus suppressing the necessity of magic-states distillation. The diagram has a higher number of nodes and edges, so the obtention of its topological depth requires the algorithmic approach for the orientation of vertical edges presented in the section of practical problem. This case, as well as the other eight diagrams, constitute trivial cases for the execution of the algorithm, as there are no vertical edges connecting nodes with the same depth.

We can build some intuition of the method by looking at figure 25. For the trivial cases where $L_{in}(u) = L_{in}(v)$, we just need to draw the vertex in an explicit layer distribution, so each of the vertical edges will be drawn as a diagonal and will be oriented in the direction of time flux.

The next step is to perform the simplification routine on each diagram. The simple execution of `full_reduce` produces graphs without possible translation to lattice surgery operations, so we once again use the escalonated simplification routine. We then select the last intermediate optimization that admits an interpretation as a physical realization of the protocol. The result for the diagram in figure 24 can ben seen in figure 26.

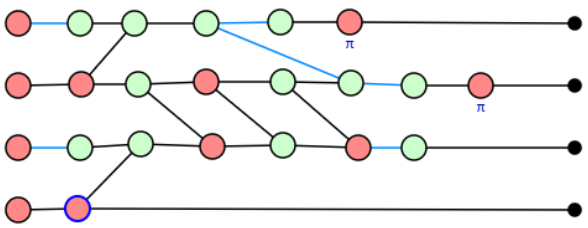

Figure 25: Visual representation of the orientation of edges for row 3 and column 0. Time flows from left to right.

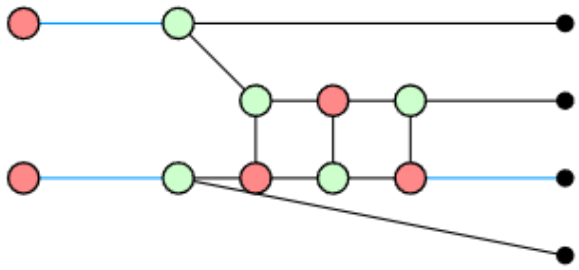

Figure 26: Simplified graph for row 3 and column 0.

We can compare the computational cost metrics for the primitive and the optimized diagrams. The data in table 1 shows that the simplification method reduces the depth of our diagrams in average around an 18%, and the pathdwidth is reduced on average approximately in a 9%. Out of the 9 diagrams, 7 of them had a reduced depth once they were simplified, and 5 of them reduced their pathwidth. All the graphs improved at least one of the metrics after the optimization.

Table 1

Metric	**(0,0)**	**(0,1)**	**(0,2)**	**(1,0)**	**(1,1)**	**(1,2)**	**(2,0)**	**(2,1)**	**(2,2)**
Original Depth	7	6	8	7	6	8	8	8	8
Original Pathwidth	6	6	6	6	6	6	6	6	7
Optimized Depth	5	6	6	6	6	6	6	6	7
Optimmized Pathwidth	5	5	6	5	5	6	6	6	6

References

[1] Frank Arute, Kunal Arya, and Ryan et al. Babbush. "Quantum supremacy using a programmable superconducting processor". In: *Nature* 574.574 (2019).

[2] Niel de Beaudrap and Dominic Horsman. "The ZX calculus is a language for surface code lattice surgery". In: *Quantum* 4 (2020). arXiv:1704.08670, p. 218.

[3] H. Bombin. *An Introduction to Topological Quantum Codes*. 2013. arXiv: 1311.0277 [quant-ph]. URL: https://arxiv.org/abs/1311.0277.

[4] S. B. Bravyi and A. Yu. Kitaev. *Quantum codes on a lattice with boundary*. 1998. arXiv: quant-ph/9811052 [quant-ph]. URL: https://arxiv.org/abs/quant-ph/9811052.

[5] John F. Clauser et al. "Proposed Experiment to Test Local Hidden-Variable Theories". In: *Phys. Rev. Lett.* 23 (15 Oct. 1969), pp. 880–884. DOI: 10.1103/PhysRevLett.23.880. URL: https://link.aps.org/doi/10.1103/PhysRevLett.23.880.

[6] Nicolas Delfosse, Pavithran Iyer, and David Poulin. *Generalized surface codes and packing of logical qubits*. 2016. arXiv: 1606.07116 [quant-ph]. URL: https://arxiv.org/abs/1606.07116.

[7] Bryan Eastin and Emanuel Knill. "Restrictions on Transversal Encoded Quantum Gate Sets". In: *Physical Review Letters* 102 (2009). arXiv:0811.4262, p. 110502.

[8] Austin G. Fowler et al. "Surface codes: Towards practical large-scale quantum computation". In: *Physical Review A* 86 (3 2012). arXiv:1208.0928, p. 032324.

[9] Steven M. Girvin. "Introduction to quantum error correction and fault tolerance". In: *SciPost Physics Lecture Notes* 70 (2023). DOI: 10.21468/SciPostPhysLectNotes.70.

[10] Daniel Gottesman. "Stabilizer Codes and Quantum Error Correction". arXiv:quant-ph/9705052. PhD thesis. California Institute of Technology, 1997.

[11] Bence Hetényi and James R. Wootton. "Creating Entangled Logical Qubits in the Heavy-Hex Lattice with Topological Codes". In: *PRX Quantum* 5.4 (Dec. 2024). ISSN: 2691-3399. DOI: 10.1103/prxquantum.5.040334. URL: http://dx.doi.org/10.1103/PRXQuantum.5.040334.

[12] Aleks Kissinger and John van de Wetering. "PyZX: Large Scale Automated Diagrammatic Reasoning". In: *Electronic Proceedings in Theoretical Computer Science* 318 (May 2020), pp. 229–241. ISSN: 2075-2180. DOI: 10.4204/eptcs.318.14. URL: http://dx.doi.org/10.4204/EPTCS.318.14.

[13] Aleks Kissinger and John van de Wetering. "Reducing the number of non-Clifford gates in quantum circuits". In: *Phys. Rev. A* 102 (2 Aug. 2020), p. 022406. DOI: 10.1103/PhysRevA.102.022406. URL: https://link.aps.org/doi/10.1103/PhysRevA.102.022406.

[14] Aleks Kissinger and John van de Wetering. "Simulating quantum circuits with ZX-calculus reduced stabiliser decompositions". In: *Quantum Science and Technology* 7.4 (2022), p. 044001. ISSN: 2058-9565. DOI: 10.1088/2058-9565/ac5d20. URL: http://dx.doi.org/10.1088/2058-9565/ac5d20.

[15] A. Yu. Kitaev. "Fault-tolerant quantum computation by anyons". In: *Annals of Physics* 303.1 (2003). arXiv:quant-ph/9707021, pp. 2–30.

[16] Emanuel Knill, Raymond Laflamme, and Lorenza Viola. "Theory of Quantum Error Correction for General Noise". In: *Physical Review Letters* 84.11 (Mar. 2000), pp. 2525–2528. ISSN: 1079-7114. DOI: 10.1103/physrevlett.84.2525. URL: http://dx.doi.org/10.1103/PhysRevLett.84.2525.

[17] Amine Laghaout et al. "A demonstration of contextuality using quantum computers". In: *European Journal of Physics* 43.5 (2022), p. 055401. ISSN: 1361-6404. DOI: 10.1088/1361-6404/ac79e0. URL: http://dx.doi.org/10.1088/1361-6404/ac79e0.

[18] William S. Massey. *Algebraic topology : an introduction*. Vol. 56. Graduate Texts in Mathematics. New York: Springer, 1967. ISBN: 3540902716 9783540902713 0387902716 9780387902715. URL: https://www.worldcat.org/title/algebraic-topology-an-introd/oclc/74404091&referer=brief_results.

[19] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. 10th Anniversary. Cambridge University Press, 2010.

[20] Alan Robertson, Haowen Gao, and Yuval R. Sanders. *A Resource Allocating Compiler for Lattice Surgery*. 2025. arXiv: 2506.04620 [quant-ph]. URL: https://arxiv.org/abs/2506.04620.

[21] Matthew Sutcliffe and Aleks Kissinger. "Fast Classical Simulation of Quantum Circuits via Parametric Rewriting in the ZX-Calculus". In: *Electronic Proceedings in Theoretical Computer Science* 426 (Aug. 2025), pp. 247–269. ISSN: 2075-2180. DOI: 10.4204/eptcs.426.10. URL: http://dx.doi.org/10.4204/EPTCS.426.10.

[22] A.A. V. *Data Structures and Algorithms*. Addison-Wesley series in computer science and information processing. Pearson Education, 1983. ISBN: 9788177588262. URL: https://books.google.es/books?id=al0wGpxm-3UC.

[23] Vivien Vandaele. *Qubit-count optimization using ZX-calculus*. 2024. arXiv: 2407.10171 [quant-ph]. URL: https://arxiv.org/abs/2407.10171.

[24] John Watrous. *Understanding Quantum Information and Computation*. 2025. arXiv: 2507.11536 [quant-ph]. URL: https://arxiv.org/abs/2507.11536.

[25] Junyu Zhou et al. *TopoLS: Lattice Surgery Compilation via Topological Program Transformations*. 2026. arXiv: 2601.23109 [quant-ph]. URL: https://arxiv.org/abs/2601.23109.